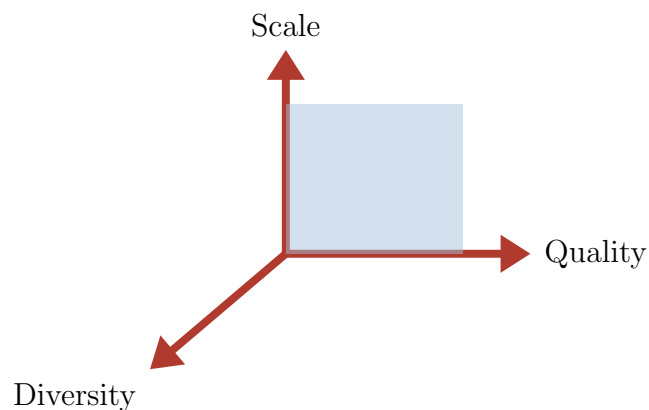

Data for Pre-training LLMs

Scaling data size · Data sources · Pre-processing pipelines · Data mixing

Detailed study notes for **Week 8.2** (Lecture 7) of
Introduction to Large Language Models — Prof. Mitesh M. Khapra
AI4Bharat, Dept. of CSE, IIT Madras



What's inside: intuition for every idea, all formulas with derivations and unit annotations, TikZ redraws of every slide figure (plus extra figures), and at the end a problem set: numerical problems with full worked solutions, subjective questions with model answers, and MCQs with explanations.

Contents

1	Orientation: where this lecture fits	3
1.1	Why a whole lecture on data?	3
1.2	Words vs. tokens vs. bytes: the units of “data size”	3
1.3	The big picture: the data pipeline (lecture roadmap)	4
1.4	Notation used throughout	4
2	Module 1: Motivation for scaling the data size	4
2.1	Evidence #1: repeating data causes overfitting (the T5 study)	5
2.2	Evidence #2: the scaling law (Kaplan et al., 2020)	5
2.3	Worked example from the slide: $N = 100\text{M}$, $D = 10\text{M}$	7
2.4	What happens when we increase D ? (the slider demo)	8
2.5	Scaling parameters: models vs. brains	9
2.6	Scaling datasets proportionately: the dataset timeline	10
2.7	The three axes of data	11
2.8	The pipeline: from 100 PB to a few TB	11
3	Module 2: Data sourcing and cleaning pipeline	11
3.1	Disclaimer: how much do we actually know?	11
3.2	Sourcing: crawling the web and Common Crawl	12
3.3	The Common Crawl data format: WARC, WAT, WET	12
3.4	What a pre-processing pipeline must do (5 mechanisms)	13
3.4.1	Language identification (LID)	14
3.4.2	Toxicity detection	14
3.4.3	Quality detection	14
3.4.4	PII removal	14
3.5	Types of deduplication	15
3.5.1	Exact dedup (1): hashing at a fixed granularity	15
3.5.2	Exact dedup (2): suffix arrays for repeated <i>substrings</i>	15
3.5.3	Fuzzy dedup (1): shingles and Jaccard similarity	16
3.5.4	Fuzzy dedup (2): MinHash	16
3.5.5	Fuzzy dedup (3): Locality Sensitive Hashing (LSH) with banding	17
3.5.6	Fuzzy dedup (4): SimHash	17
3.6	A basic pipeline (fastText)	18
3.7	The CCNet pipeline	18
3.7.1	Motivation: low-resource languages	18

3.7.2	How CCNet works	19
3.7.3	CCNet output: tokens per language	20
3.8	The Colossal Clean Crawled Corpus (C4) pipeline	20
3.8.1	The C4 heuristics (as described in the T5 paper)	21
3.8.2	Worked illustration: cleaning three “lemon” pages	21
3.9	Variations on the theme: Pile-CC	22
3.10	RefinedWeb (RW): the stringent web-only pipeline	23
3.11	Comparison of pipelines (from the RefinedWeb paper)	24
3.12	The SETU pipeline (Sangraha, AI4Bharat)	25
3.13	Perfect deduplication is difficult	26
3.14	Why is deduplication important?	26
3.15	All pipelines are leaky!	27
4	Module 3: Pre-training datasets	28
4.1	Availability problems	28
4.2	A taxonomy of pre-training datasets	28
4.3	How to compare dataset sizes	29
4.4	The datasets at a glance	29
4.5	Composite datasets	30
4.6	Data diversity and data mixing	31
4.6.1	Real mixtures: GPT-3, LLaMA, PaLM	32
4.7	Represented domains across models	33
4.8	Some questions to ponder (with discussion)	34
4.9	References (from the slides)	34
5	One-page summary (revision sheet)	36
6	Numerical problems with full worked solutions	37
7	Subjective questions with model answers	44
8	Multiple-choice questions	47
8.1	Answer key with explanations	52

1 Orientation: where this lecture fits

1.1 Why a whole lecture on data?

Up to now the course has been about *architectures*: the transformer, encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5), and objectives like masked LM, causal LM and span corruption. This lecture changes the question. Instead of “*what network do we train?*” it asks “*what do we train it on, how much of it, and how do we clean it?*”

The lecture opens with a table of the datasets we have met so far:

Model	Dataset	Size	Parameters
GPT	BookCorpus	0.8 billion words	117M
BERT	BookCorpus + English Wikipedia	3.3 billion words	110M / 340M
GPT-2	WebText	~19 billion tokens	1.5B
T5	C4	~175 billion tokens	up to 11B (slide: “10B”)

Notice the jump. Going from GPT-2 (1.5B parameters, 19B tokens) to T5 (about 10B parameters, 175B tokens) grows the parameter count about $7\times$ and the token count about $9\times$. The lecture poses three questions, and they organise everything that follows:

- Q1.** Is it necessary to scale the size of the dataset even further? → **Module 1**
- Q2.** If yes, how do we *source* the data? → **Module 2**
- Q3.** What aspects (duplication, toxicity, bias, ...) must we consider while building a larger dataset, and how do we mix sources? → **Modules 2 & 3**

Intuition — the “fuel” analogy

If the model is an engine, data is the fuel. Module 1 says a bigger engine needs *more* fuel, or it floods (overfits). Module 2 says raw fuel (the web) is dirty, so we refine it. Module 3 says different fuels (web, books, code, papers) give the engine different abilities, so we blend them in chosen proportions.

1.2 Words vs. tokens vs. bytes: the units of “data size”

The slides measure data size in three different units. Mixing them up is a common exam mistake.

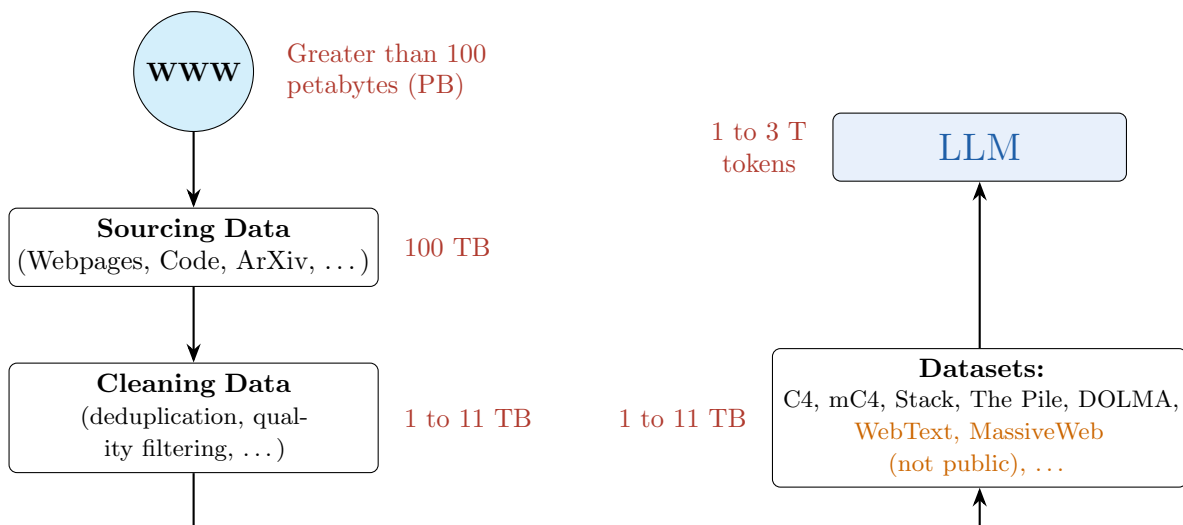
Unit	Meaning and typical conversion
Words	Whitespace-separated words. Used for BookCorpus/BERT. For English, 1 word $\approx$ 1.3 BPE tokens.
Tokens	The unit the model actually consumes (BPE / SentencePiece / WordPiece pieces). This is the D that appears in scaling laws.
Bytes (GB/TB)	Disk size of the (uncompressed) text. English web text is $\approx$ 4–5 bytes per token. For example C4: ~ 750 GB $\leftrightarrow$ ~ 156 B tokens gives $\approx$ 4.8 bytes/token.
GiB/TiB	Binary units: 1 GiB = 2^{30} bytes $\approx$ 1.074×10^9 bytes; 1 TiB = 2^{40} bytes $\approx$ 1.0995×10^{12} bytes.

Careful — the same dataset has different “sizes”

Token counts depend on the tokenizer. The Pile is quoted as $\sim 170\text{B}$ tokens in one slide and as $\sim 340\text{B}$ “GT” (giga-tokens) in the RefinedWeb comparison table. Different tokenizers and counting conventions (with or without epochs/upsampling) give different numbers. In an exam, use the number the question gives you.

1.3 The big picture: the data pipeline (lecture roadmap)

The single most important figure of the lecture is the end-to-end pipeline, which moves from $> 100\text{ PB}$ of web down to a 1–3 trillion-token training stream:

**Key idea — the funnel**

Each stage removes a large fraction. $100\text{ PB} \rightarrow 100\text{ TB}$ is a factor of 10^3 (we only fetch the parts we want). $100\text{ TB} \rightarrow 1\text{--}11\text{ TB}$ is another factor of roughly 10–100 (cleaning). Pre-training data is therefore a tiny, heavily refined slice of the web. Later (RefinedWeb) we will see a single Common Crawl snapshot lose **almost 90%** of its content during cleaning.

1.4 Notation used throughout

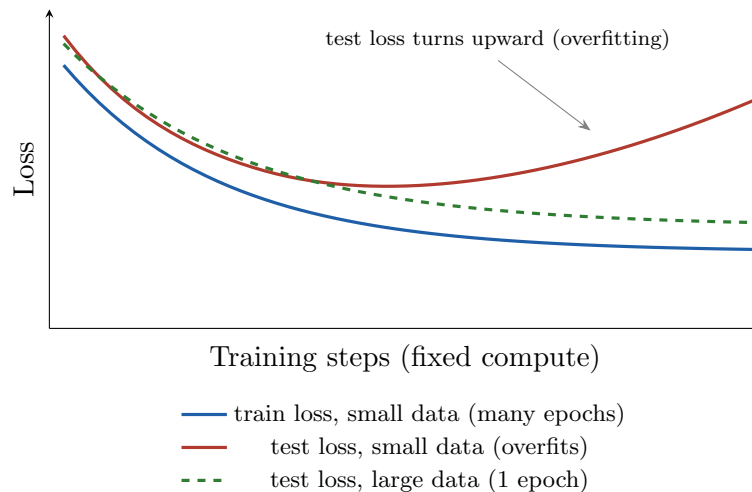
Symbol	Meaning	Units
N	number of <i>non-embedding</i> model parameters	parameters
D	dataset size	tokens
L	cross-entropy test loss per token	nats/token
N_c, D_c	fitted scale constants of the scaling law	parameters, tokens
α_N, α_D	fitted scaling exponents	dimensionless
$J(A, B)$	Jaccard similarity of sets A, B	dimensionless, $\in [0, 1]$
p_i	sampling proportion of source i in the mixture	dimensionless, $\sum_i p_i = 1$
T	total training tokens	tokens

2 Module 1: Motivation for scaling the data size

2.1 Evidence #1: repeating data causes overfitting (the T5 study)

The T5 paper (“Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”) ran a controlled experiment. Keep the *number of training steps* fixed, but shrink the pre-training corpus, so that the model has to cycle over the same data for more and more epochs.

Finding: the model *overfits* when data is repeated for many epochs, and downstream performance degrades. The conclusion on the slide: *we should scale the data size rather than re-use a small dataset.*



Schematic (not data from the paper): with a small corpus seen many times, training loss keeps falling while test loss turns upward.

Intuition — why repetition overfits

A model with billions of parameters has enough capacity to store specific sequences. The first time it sees a document it must *generalise* to predict it well. After the tenth pass it can simply *recall* it. Gradient updates then push the weights toward memorising those exact strings instead of learning transferable patterns. Fresh data forces generalisation on every step.

Beyond the slides — what the T5 experiment actually varied

T5 pre-trained for a fixed $2^{35} \approx 34\text{B}$ tokens. They truncated C4 to $2^{29}, 2^{27}, 2^{25}, 2^{23}$ tokens, which forces 64, 256, 1024, 4096 repetitions. Performance was roughly unchanged at 64 repeats, then dropped as repeats increased, and the training loss fell suspiciously low, a signature of memorisation. $\text{Epochs} = \frac{\text{tokens trained}}{\text{dataset tokens}}$; e.g. $2^{35}/2^{29} = 2^6 = 64$.

2.2 Evidence #2: the scaling law (Kaplan et al., 2020)

A few months after T5, the “Scaling Laws for Neural Language Models” study gave a quantitative reason to scale data. Given N non-embedding parameters and D training tokens, the **test** loss is well fitted by

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \quad (1)$$

with fitted constants

$$\begin{aligned} N_c &= 8.8 \times 10^{13} \text{ (88 trillion parameters),} & \alpha_N &= 0.076, \\ D_c &= 5.4 \times 10^{13} \text{ (54 trillion tokens),} & \alpha_D &= 0.095. \end{aligned}$$

Dimension (unit) annotations

$$\underbrace{L}_{\text{nats/token}} = \left[\underbrace{\left(\frac{N_c \text{ [params]}}{N \text{ [params]}} \right)^{\alpha_N/\alpha_D}}_{\text{dimensionless}} + \underbrace{\frac{D_c \text{ [tokens]}}{D \text{ [tokens]}}}_{\text{dimensionless}} \right]^{\alpha_D \text{ [dimensionless]}}$$

Both terms inside the bracket are pure numbers, so they can be added. N_c and D_c exist *exactly* to make the ratios unit-free. The loss is a cross-entropy (average negative log-likelihood per token). With natural logs it is in nats; the constants were fitted that way.

Why “non-embedding” parameters?

The embedding matrix has $|V| \times d_{\text{model}}$ parameters, and for small models it can dominate the count while contributing little “compute”. Kaplan et al. found much cleaner power laws when they excluded embedding (and positional-embedding) parameters. For a transformer with n_{layer} layers and width d_{model} , the non-embedding count is approximately

$$N \approx 12 n_{\text{layer}} d_{\text{model}}^2$$

($4d^2$ for the attention projections W_Q, W_K, W_V, W_O , plus $8d^2$ for the MLP with $d_{ff} = 4d$), ignoring biases and LayerNorm.

Derivation — the two limiting regimes of Eq. (1)

(a) **Infinite data**, $D \rightarrow \infty$. Then $D_c/D \rightarrow 0$ and

$$L(N, \infty) = \left[\left(\frac{N_c}{N} \right)^{\alpha_N/\alpha_D} \right]^{\alpha_D} = \left(\frac{N_c}{N} \right)^{\alpha_N}.$$

This is the pure *model-size-limited* law $L(N) = (N_c/N)^{\alpha_N}$. It is the best loss a size- N model can reach.

(b) **Infinite model**, $N \rightarrow \infty$. Then $(N_c/N)^{\alpha_N/\alpha_D} \rightarrow 0$ and

$$L(\infty, D) = \left(\frac{D_c}{D} \right)^{\alpha_D},$$

the pure *data-limited* law. Even an infinitely large model is bottlenecked by how much data it saw.

Why the odd exponent α_N/α_D ? It is chosen *so that* both limits come out as clean power laws. Raising $(N_c/N)^{\alpha_N/\alpha_D}$ to the power α_D gives back $(N_c/N)^{\alpha_N}$. Numerically $\alpha_N/\alpha_D = 0.076/0.095 = 0.8$, which is why the slide writes the exponent as 0.8.

Derivation — how much does more data help? (elasticity)

Let $A = (N_c/N)^{0.8}$ and $B = D_c/D$. Then $L = (A+B)^{\alpha_D}$ and $\ln L = \alpha_D \ln(A+B)$. Differentiate w.r.t. $\ln D$, noting $\frac{dB}{d \ln D} = -B$:

$$\frac{d \ln L}{d \ln D} = \alpha_D \cdot \frac{-B}{A+B} = -\alpha_D \frac{D_c/D}{(N_c/N)^{0.8} + D_c/D}.$$

- If data is the bottleneck ($B \gg A$), $\frac{d \ln L}{d \ln D} \approx -\alpha_D = -0.095$. Every doubling of data multiplies the loss by $2^{-0.095} \approx 0.936$, a 6.4% drop.
- If the model is the bottleneck ($A \gg B$), $\frac{d \ln L}{d \ln D} \approx 0$. More data barely helps; you need a bigger model.

Interpretation: “the more the data, the better the model” holds *until* the data term becomes small compared with the model term.

Derivation — how should D grow with N to avoid overfitting?

Compare the finite-data loss with the infinite-data loss of the same model:

$$\frac{L(N, D)}{L(N, \infty)} = \frac{(A+B)^{\alpha_D}}{A^{\alpha_D}} = \left(1 + \frac{B}{A}\right)^{\alpha_D}, \quad \frac{B}{A} = \frac{D_c}{D} \left(\frac{N}{N_c}\right)^{0.8}.$$

The “overfitting penalty” stays constant if B/A stays constant, i.e.

$$D \propto N^{\alpha_N/\alpha_D} = N^{0.8}.$$

Meaning: when you make the model $10\times$ larger, the data must grow by $10^{0.8} \approx 6.3\times$ to keep the same relative overfitting. *Scaling parameters requires scaling data proportionally*; this is exactly the “Scaling Dataset proportionately” box on the timeline slides. (Kaplan et al. state the empirical rule as $D \gtrsim 5 \times 10^3 N^{0.74}$; the 0.8 here comes straight from Eq. (1).)

2.3 Worked example from the slide: $N = 100\text{M}$, $D = 10\text{M}$

Question: a model has $N = 100$ million non-embedding parameters and is trained on $D = 10$ million tokens. What is the training loss? What does the scaling law predict for the test loss?

Training loss. A 100M-parameter model trained on only 10M tokens can memorise the data completely. There are 10 parameters per training token. The training loss can be driven to (nearly) **zero**.

Test loss. The scaling law describes generalisation, so

$$\begin{aligned} L &= \left[\left(\frac{88 \times 10^{12}}{100 \times 10^6} \right)^{0.8} + \frac{54 \times 10^{12}}{10 \times 10^6} \right]^{0.095} \\ &= \left[(8.8 \times 10^5)^{0.8} + 5.4 \times 10^6 \right]^{0.095}. \end{aligned}$$

Step by step with logarithms:

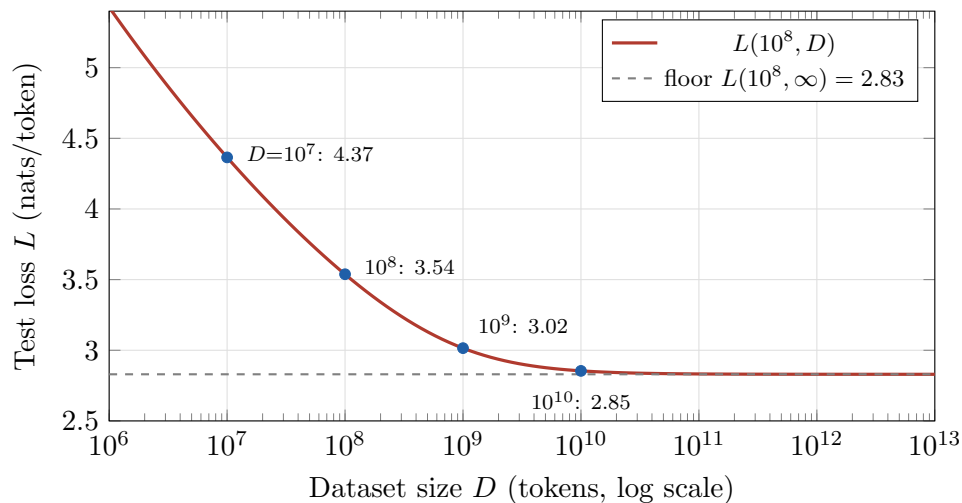
$$\begin{aligned} \log_{10}(8.8 \times 10^5) &= 5.9445 \Rightarrow 0.8 \times 5.9445 = 4.7556 \Rightarrow 10^{4.7556} \approx 56\,962, \\ 56\,962 + 5\,400\,000 &= 5\,456\,962 \Rightarrow \log_{10} = 6.7369, \\ 0.095 \times 6.7369 &= 0.6400 \Rightarrow L = 10^{0.6400} \approx \boxed{4.36 \text{ nats/token}}. \end{aligned}$$

Intuition — read the two terms

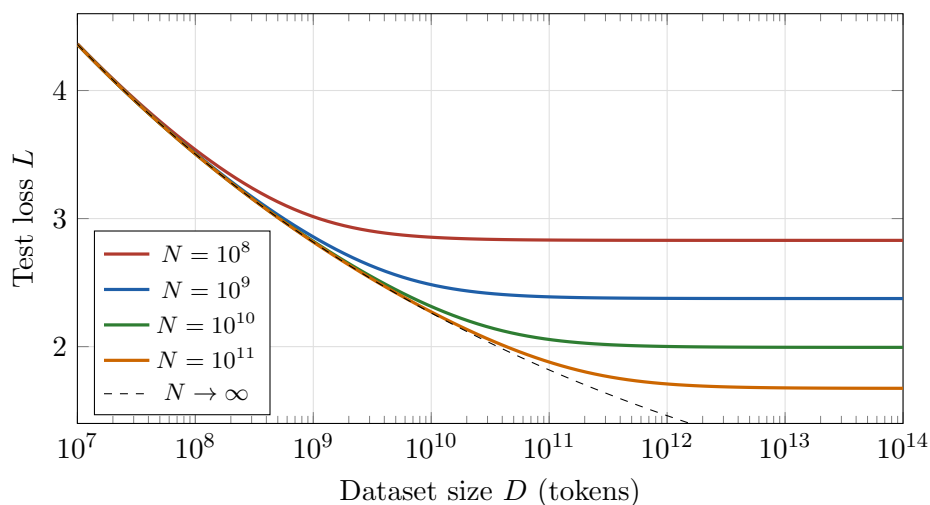
The model term is $\approx 5.7 \times 10^4$ and the data term is 5.4×10^6 . The data term is about $95\times$ larger, so this model is **severely data-starved**. Its huge gap between train loss (≈ 0) and test loss (4.36) is overfitting in its purest form. The remedy is more tokens, not more parameters.

2.4 What happens when we increase D ? (the slider demo)

In the lecture, a GeoGebra slider increases D in steps of 10M tokens. The figure below is the same curve, computed from Eq. (1), for $N = 10^8$:

**Observations:**

1. Increasing the number of tokens **reduces the test loss**, the slide's main message.
2. The curve **saturates** at $L(N, \infty) = (N_c/N)^{\alpha_N} = 2.83$. Beyond $\sim 10^{10}$ tokens, this 100M model can no longer use extra data. It is now *capacity-limited*.
3. Hence the community's conclusion: **scale both model and data together** [Kaplan et al.].



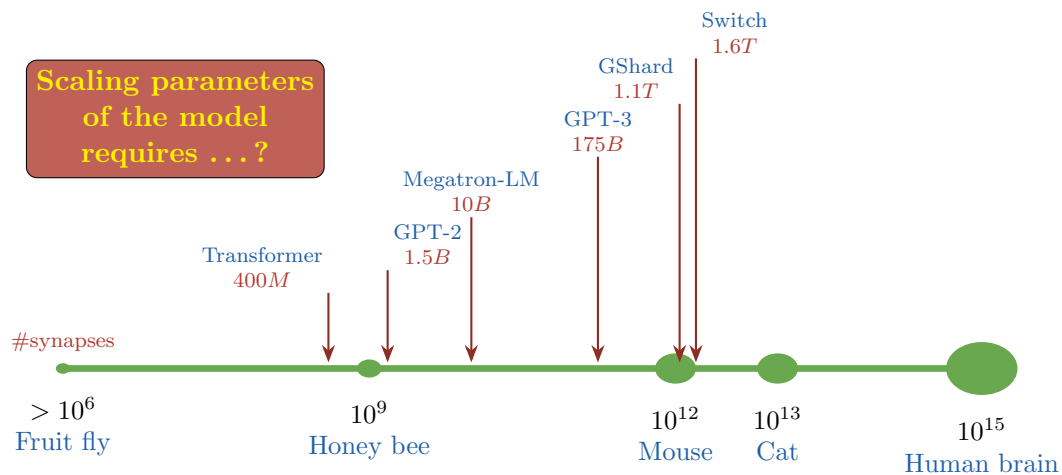
Extra figure: a family of curves. Bigger models reach lower floors but need exponentially more data to get there. All curves hug the $N \rightarrow \infty$ envelope while data-limited.

Exam tip

If asked “will more data always reduce test loss?”, the correct answer is: *yes, monotonically* ($\partial L / \partial D < 0$ always), *but with diminishing returns*. The loss approaches the floor $(N_c/N)^{\alpha_N}$ set by the model size.

2.5 Scaling parameters: models vs. brains

The lecture places model sizes on a log axis next to the number of *synapses* in biological brains (a loose, popular comparison, not a claim of equivalence):

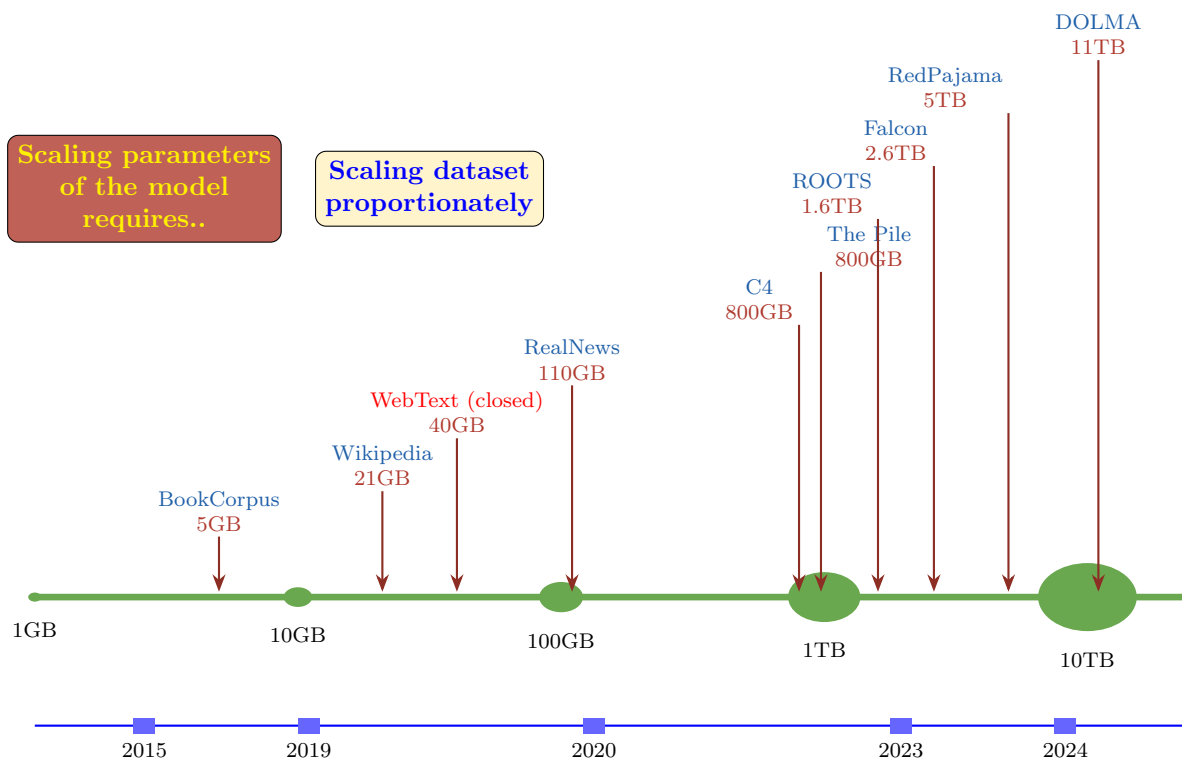


Parameter counts grew from 400M (the 2017 Transformer, big variant) to 1.6T (Switch Transformer, a sparse Mixture-of-Experts) in about four years. The question on the slide: *what does scaling parameters require?* Answer from the previous subsection: **proportionally scaled data**.

Careful — dense vs. sparse

GShard (600B in its paper; 1.1T in later variants) and Switch (1.6T) are *Mixture-of-Experts* models. Only a small fraction of parameters is active per token, so their “size” is not directly comparable with dense GPT-3 (175B). The synapse comparison is a motivational picture, not neuroscience.

2.6 Scaling datasets proportionately: the dataset timeline



Datasets grew from ~5 GB (BookCorpus, 2015) to ~11 TB (DOLMA, 2024), more than $2000\times$ in under a decade. WebText (OpenAI, used for GPT-2) was never released (“closed”); the community recreated it as OpenWebText.

Three trends in the literature

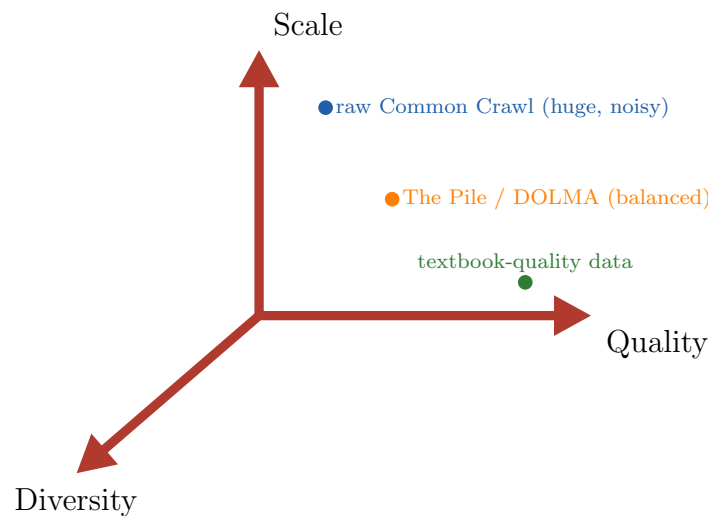
While data has been growing, three themes emerged (each tied to a paper on the slide):

Emphasizing diversity <i>The Pile</i>: an 800GB dataset of diverse text (22 sources) <i>RedPajama-Data-v2</i>: an open dataset with 30 trillion tokens	Data quality is super important <i>RefinedWeb</i> (Falcon): well-processed web data <i>alone</i> beats curated corpora <i>Textbooks Are All You Need</i>: gains along the <i>quality</i> axis	Open-source data is good enough <i>LLaMA</i>: state-of-the-art performance training <i>exclusively</i> on publicly available data
--	---	--

Intuition — these are not contradictory

Diversity says “*cover many kinds of text*”. Quality says “*each kind should be clean and informative*”. Open data says “*you don’t need secret data to do it*”. A good dataset does all three. RefinedWeb seems to challenge diversity: it shows that the web *itself* is diverse enough when filtered well.

2.7 The three axes of data



- **Scale:** how many tokens (Module 1: needed by the scaling law).
- **Quality:** how clean, informative, non-toxic, non-duplicated (Module 2: the cleaning pipeline).
- **Diversity:** how many domains, languages, styles (Module 3: composite datasets and mixing).

Key idea — the trade-off

The axes pull against each other. Aggressive quality filtering shrinks *scale* (RefinedWeb discards ~90%). Filtering against a narrow “gold” reference can also shrink *diversity* (Gopher’s warning, Module 2). Dataset design means choosing a point in this 3-D space.

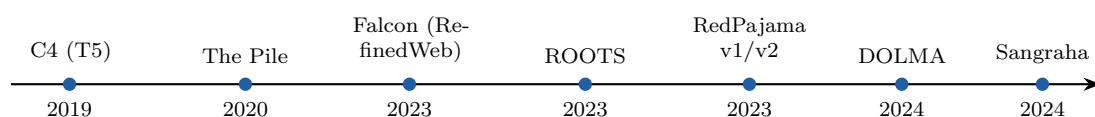
2.8 The pipeline: from 100 PB to a few TB

The overview figure in Section 1.3 is the roadmap for Module 2. The web (> 100 PB) is *sourced* (~ 100 TB of webpages, code, arXiv, ...), *cleaned* (dedup, quality filtering, ...) to 1–11 TB, published as datasets (C4, mC4, the Stack, the Pile, DOLMA; some private, like WebText and MassiveWeb), and finally consumed as 1–3 trillion tokens by the LLM.

3 Module 2: Data sourcing and cleaning pipeline

3.1 Disclaimer: how much do we actually know?

Detailed, public documentation of the data-cleaning pipeline exists for only a handful of datasets:



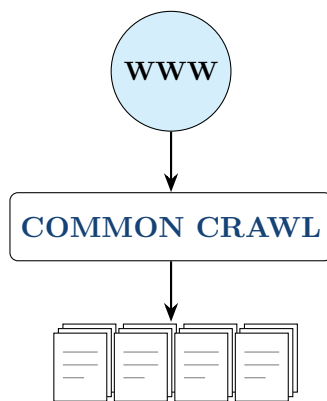
- Datasets used to pre-train large models often **lack proper documentation**. Technical reports give only very basic information.

- *Example: MassiveWeb*, used to train DeepMind’s **Gopher**, is described only as using a *custom HTML scraper* with *Google’s SafeSearch signal* for filtering. The data and the exact process are not public.
- The lecture therefore aims to give a *comprehensive idea* of the pipelines that *are* documented. They share a common skeleton, so understanding them covers most of what you need.

3.2 Sourcing: crawling the web and Common Crawl

Almost all pre-training datasets are built by **scraping/crawling the web**, which holds more than **100 petabytes** of data. There are two ways to get it:

1. Write a **custom HTML scraper** (e.g. MassiveWeb, WebText). This gives full control but is expensive and hard to reproduce.
2. Use **Common Crawl (CC)**, a non-profit that crawls the web and releases the data for free. Each month it releases a **snapshot** of about **3 to 5 billion web pages**.



Monthly snapshot: 3–5 billion pages.

Raw data contains **duplicate, toxic, low-quality** content and **personally identifiable information (PII)**.

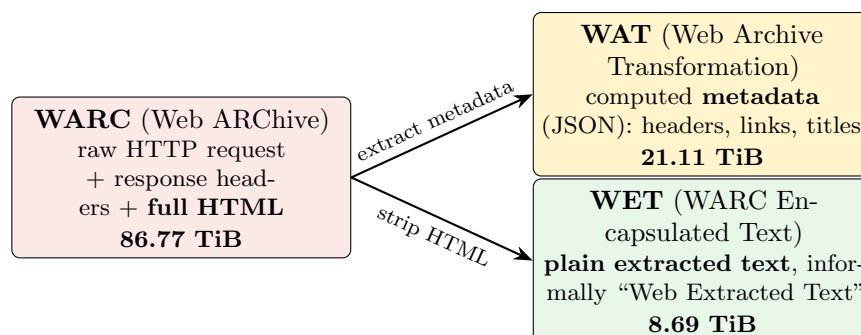
Garbage in – Garbage out (GIGO), so we **must** clean the raw text.

Intuition — GIGO for language models

A language model learns to imitate the distribution of its training text. If 5% of that text is cookie banners, spam, or offensive rants, the model learns to produce them, and wastes capacity doing so. Cleaning is not cosmetic. It decides what “normal language” means to the model.

3.3 The Common Crawl data format: WARC, WAT, WET

Raw CC data comes in three formats (sizes are for one snapshot, compressed):



Data type	#Files	Total size compressed (TiB)
Segments	100	—
WARC	80 000	86.77
WAT	80 000	21.11
WET	80 000	8.69
Robots.txt files	80 000	0.14
Non-200 responses	80 000	2.34
URL index files	302	0.23
Columnar URL index files	900	0.26

Key idea — which format should a pipeline start from?

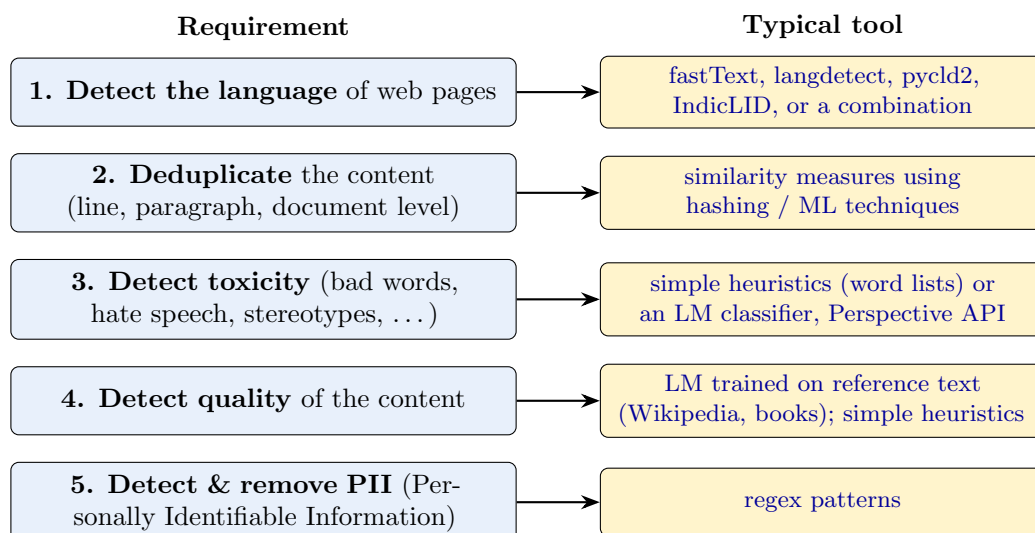
- **WET** is ready-made text, only $\approx 8.69/86.77 \approx 10\%$ of the WARC size. It is cheap, but CC's own text extraction is crude: menus, footers and boilerplate stay in. Used by CCNet and C4.
 - **WARC** holds the raw HTML, so *you* choose the extractor (jusText, Trafilatura). It is costly but gives much cleaner text. Used by Pile-CC, RefinedWeb and Sangraha.
- The pipeline takes raw data in any of these formats and outputs **clean text**.

Challenges.

1. Large datasets use CC snapshots from **multiple months or years**, i.e. hundreds of TiB, so the pipeline must be **automatic** and massively parallel.
2. It is hard to **directly measure how clean** the output is. There is no ground-truth “clean” label for billions of pages. We usually judge cleaning indirectly, by training models on the result and evaluating them.

3.4 What a pre-processing pipeline must do (5 mechanisms)

To get high-quality text from raw data, the pipeline needs mechanisms to:



We now open each box.

3.4.1 Language identification (LID)

fastText LID: a linear classifier over bag-of-character- n -gram embeddings. For a document x with n -gram set $G(x)$:

$$\mathbf{h}(x) = \frac{1}{|G(x)|} \sum_{g \in G(x)} \mathbf{e}_g \quad [\mathbf{e}_g, \mathbf{h}(x) \in \mathbb{R}^d, d \approx 16]$$

$$P(\ell \mid x) = \text{softmax}(\mathbf{W}\mathbf{h}(x))_\ell \quad [\mathbf{W} \in \mathbb{R}^{|\mathcal{L}| \times d}, |\mathcal{L}| = 176 \text{ languages}]$$

A page is assigned language $\hat{\ell} = \arg \max_{\ell} P(\ell \mid x)$ and *kept* only if $P(\hat{\ell} \mid x) \geq \tau$. CCNet uses $\tau = 0.5$; RefinedWeb uses 0.65. C4 used **langdetect** and kept pages with probability ≥ 0.99 of being English.

Intuition — why character n -grams?

Languages have fingerprints at the sub-word level: “-tion”, “sch”, “-ment”. Character n -grams catch them even for unseen words, and they are cheap enough to run on billions of pages. **IndicLID** (AI4Bharat) is specialised for 22 Indian languages, including *romanised* text such as Hindi written in Latin script, which generic LIDs misclassify as English.

3.4.2 Toxicity detection

- **Heuristics**: a manual list of bad words. Drop any page that contains one (C4). It is cheap and transparent but blunt: it also removes medical or LGBTQ+ content that merely mentions certain words.
- **LM/ML classifier**: a model scores toxicity (e.g. **Perspective API** by Jigsaw/Google returns a probability that a text is toxic). It is more nuanced, but it can carry the classifier’s own biases.

3.4.3 Quality detection

- **Reference-LM approach**: train an LM (or a classifier) on a “high-quality” reference corpus (Wikipedia, books). Pages that look like the reference get low perplexity or a high classifier score and are kept.
- **Heuristics**: rules such as “lines must end in punctuation” or “not too many symbols”.

Derivation — perplexity as a quality score

For a document $w_1, \dots, w_T$ and a reference LM q (e.g. a 5-gram KenLM trained on Wikipedia),

$$\text{PPL}(w_{1:T}) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \ln q(w_t \mid w_{<t})\right) \quad [\text{dimensionless}, \geq 1].$$

Here $-\frac{1}{T} \sum \ln q$ is the average cross-entropy in nats/token, and exponentiating gives the “effective branching factor”, i.e. how many equally likely choices the reference LM sees per token. **Low PPL** means the text looks like Wikipedia and counts as *higher quality*. **High PPL** means gibberish, lists, or boilerplate.

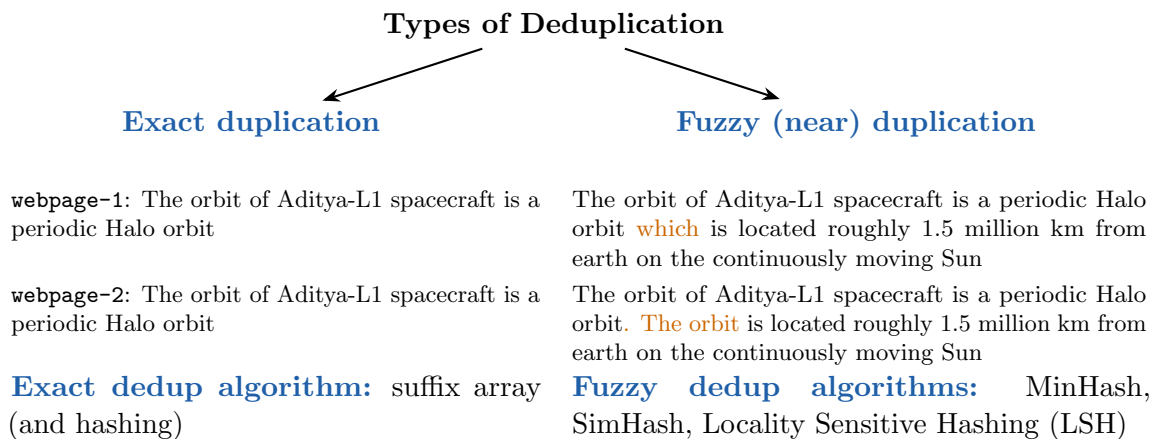
3.4.4 PII removal

Regex patterns find emails, phone numbers, IP addresses, etc. and replace them with placeholder tokens:

PII type	Illustrative regex
Email	<code>[\w.+-]+@[\w-]+\.[\w.]+</code>
IPv4 address	<code>\b(?:\d{1,3}\.){3}\d{1,3}\b</code>
Indian mobile	<code>(?:\+91[\s-]?)?[6-9]\d{9}</code>

DOLMA, for example, replaces a detected email with a token like `|||EMAIL_ADDRESS|||`. Regexes are precise for structured PII, but they cannot catch names or addresses written in free text.

3.5 Types of deduplication



Intuition

Exact duplicates are byte-identical, e.g. the same boilerplate on 10 000 pages or a copied news article. *Near* duplicates differ trivially: a changed word, different punctuation, a date stamp, a template with a different product name. Exact matching misses the second kind, and the web is full of it (mirrors, scraped copies, templated pages).

3.5.1 Exact dedup (1): hashing at a fixed granularity

Normalise each unit (lower-case, strip digits and whitespace), hash it (e.g. SHA-1 or a 64-bit hash), and keep only the first occurrence of each hash. The **unit** can be a line (fastText-style basic pipeline), a paragraph (CCNet), or a whole document.

With n units and a hash table this is $O(n)$ expected time. Memory is $O(\# \text{unique units})$, i.e. 8 bytes per 64-bit hash. For 10^{10} paragraphs that is ~ 80 GB, so it is sharded across machines.

Careful — hash collisions

With b -bit hashes and n distinct items, the expected number of colliding pairs is $\approx \binom{n}{2} 2^{-b} \approx n^2 / 2^{b+1}$. For $n = 10^{10}$ and $b = 64$ this is $10^{20} / 3.7 \times 10^{19} \approx 2.7$ false “duplicates”, which is negligible. With $b = 32$ it would be catastrophic.

3.5.2 Exact dedup (2): suffix arrays for repeated *substrings*

Hashing lines misses a long paragraph copied *inside* otherwise different documents. A **suffix array** finds every repeated substring above a length threshold (e.g. ≥ 50 tokens, as in Lee et al. 2021 and RefinedWeb).

Construction. Concatenate the whole corpus into one string S of length n . List all n suffixes and sort them lexicographically. The suffix array SA stores their starting positions in sorted order. Identical substrings become prefixes of *adjacent* suffixes, so a linear scan of the LCP (longest common prefix) array finds all repeats.

rank	$SA[i]$	suffix of banana\$	LCP with previous
0	6	\$	—
1	5	a\$	0
2	3	ana\$	1
3	1	ana na\$	3
4	0	banana\$	0
5	4	na\$	0
6	2	na na\$	2

LCP=3 between ranks 2 and 3 shows that **ana** occurs twice (positions 1 and 3). In a corpus, an $LCP \geq 50$ tokens between adjacent suffixes flags a duplicated span, and all but one copy is removed.

Complexity: $O(n \log n)$ with comparison sorting, or $O(n)$ with SA-IS. Memory is $O(n)$ integers, which is why it is expensive at web scale.

3.5.3 Fuzzy dedup (1): shingles and Jaccard similarity

Represent each document d as its set of **shingles** (k -grams of words or characters), $S(d)$. Similarity is the **Jaccard index**:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1] \quad [\text{dimensionless}].$$

For the Aditya-L1 pair (lower-cased, punctuation removed), word 3-shingles give $|A| = 22$, $|B| = 23$, $|A \cap B| = 19$ and $|A \cup B| = 26$, so $J = 19/26 \approx 0.73$. The two differ only by “which” vs. “. The orbit”. Word 1-grams give $J = 20/21 \approx 0.95$. (Full working in the Numerical Problems section.)

Computing J exactly for all $\binom{n}{2}$ pairs of billions of documents is impossible. MinHash and LSH make it feasible.

3.5.4 Fuzzy dedup (2): MinHash

Take a random hash function h that maps shingles to integers (equivalently, a random permutation of the shingle universe). Define the signature value

$$m_h(A) = \min_{s \in A} h(s).$$

Derivation — the MinHash property $P[m_h(A) = m_h(B)] = J(A, B)$

Consider the union $A \cup B$. Under a random permutation, every element of $A \cup B$ is equally likely to be the one with the smallest hash; call it s^* .

- If $s^* \in A \cap B$, it is the minimum of *both* A and B , so $m_h(A) = m_h(B)$.
- If $s^* \in A \setminus B$ (or $B \setminus A$), then $m_h(A) = h(s^*) < m_h(B)$ (or the reverse), so they differ.

Therefore

$$P[m_h(A) = m_h(B)] = P[s^* \in A \cap B] = \frac{|A \cap B|}{|A \cup B|} = J(A, B). \quad \blacksquare$$

Estimator. With k independent hash functions, the signature is $\mathbf{m}(A) = (m_{h_1}(A), \dots, m_{h_k}(A)) \in \mathbb{Z}^k$ and

$$\hat{J} = \frac{1}{k} \sum_{i=1}^k \mathbb{1}[m_{h_i}(A) = m_{h_i}(B)], \quad \mathbb{E}[\hat{J}] = J, \quad \text{Var}[\hat{J}] = \frac{J(1-J)}{k}.$$

The estimate is a Binomial proportion, so its standard error is $\sqrt{J(1-J)/k}$. For $k = 100$ and $J = 0.8$ that is 0.04. Each document shrinks from thousands of shingles to a fixed k integers (e.g. $k = 9000$ in RefinedWeb, 20 buckets $\times$ 450).

3.5.5 Fuzzy dedup (3): Locality Sensitive Hashing (LSH) with banding

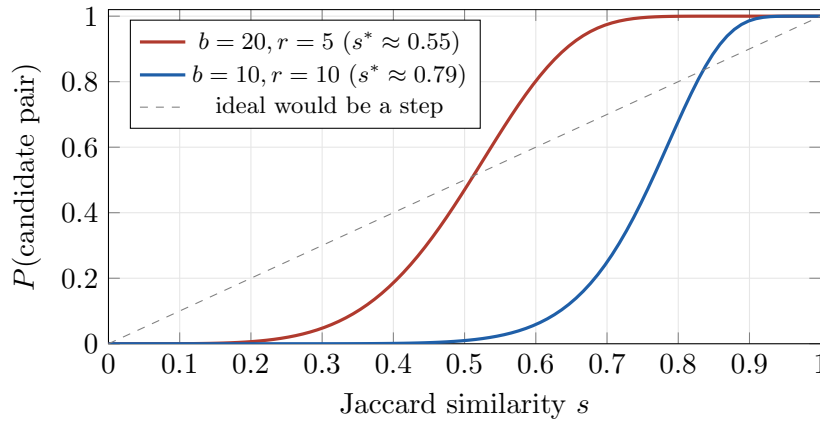
Even with signatures, comparing all pairs is $O(n^2)$. LSH splits the k -dim signature into b **bands** of r rows ($k = br$). Two documents become **candidates** if they agree on *all* r rows of *at least one* band. Documents are hashed band-by-band into buckets, so only documents that share a bucket are compared.

Derivation — the S-curve

For a pair with Jaccard s : one row matches w.p. s ; all r rows of a band match w.p. s^r ; a band fails w.p. $1 - s^r$; all b bands fail w.p. $(1 - s^r)^b$. Hence

$$P(\text{candidate} \mid s) = 1 - (1 - s^r)^b.$$

The steepest point (threshold) is at $s^* \approx (1/b)^{1/r}$. For $b = 20, r = 5$ this gives $s^* \approx 0.55$.



Intuition — the knobs

Increasing r makes a band harder to match, which cuts false positives and moves the threshold right. Increasing b gives more chances to match, which cuts false negatives and moves the threshold left. You tune (b, r) so the steep part of the S-curve sits at the similarity you call “duplicate” (often 0.7–0.8).

3.5.6 Fuzzy dedup (4): SimHash

SimHash (Charikar 2002) maps a document to an f -bit fingerprint (e.g. $f = 64$). Each feature g with weight w_g has a random hash $\mathbf{r}_g \in \{\pm 1\}^f$. Then

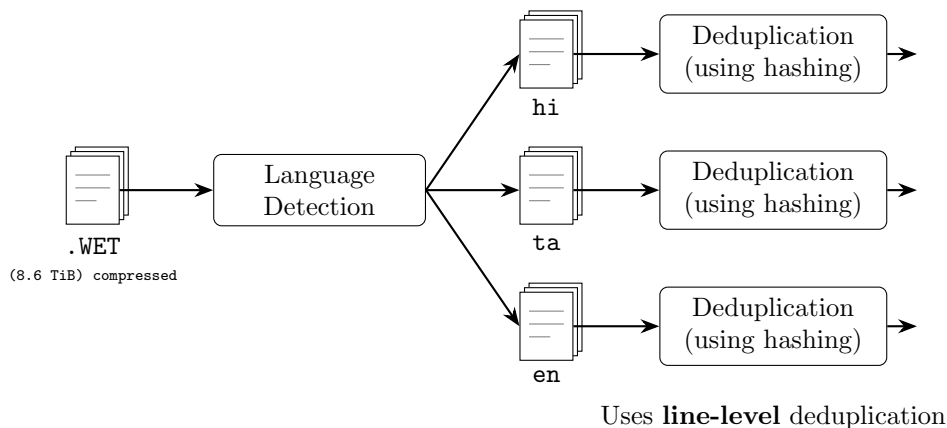
$$\text{bit}_j = \mathbb{I}\left[\sum_g w_g r_{g,j} > 0\right], \quad j = 1..f.$$

This is a random-hyperplane projection, and $P[\text{bit}_j(A) = \text{bit}_j(B)] = 1 - \theta_{AB}/\pi$, where θ_{AB} is the angle between the feature vectors (cosine similarity). Near-duplicates have fingerprints that differ in only a few bits (small **Hamming distance**, e.g. ≤ 3 out of 64). Google famously used this for web-crawl dedup.

	MinHash	SimHash	Suffix array
Detects	near-dup (Jaccard)	near-dup (cosine)	exact repeated substrings
Granularity	document	document	span (any length $\geq$ threshold)
Signature	k integers	f bits	none (index over corpus)
Scaling trick	LSH banding	bit-block tables	sort once, scan LCP
Used by	GPT-3, Pile, RefinedWeb, Sangraha	web search engines	Lee et al., RefinedWeb

3.6 A basic pipeline (fastText)

The simplest pipeline, released with fastText’s multilingual word vectors (Grave et al. 2018):



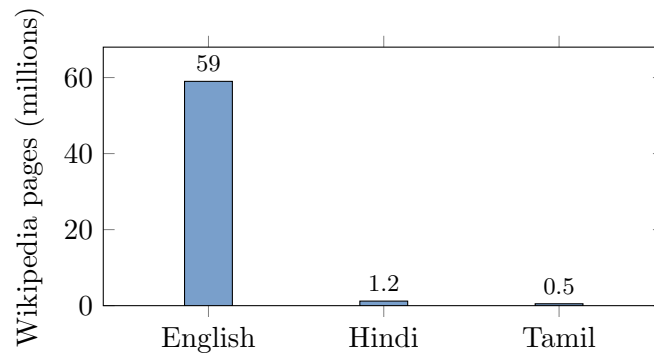
- Input: WET files. Step 1: fastText LID splits pages by language (hi, ta, en, ...). Step 2: **line-level** hash dedup within each language.
- It satisfies **only two** of the five requirements of Section 3.4 (language detection and deduplication).
- The filtered text can **still contain toxic, low-quality content and PII**.

3.7 The CCNet pipeline

3.7.1 Motivation: low-resource languages

CCNet (Wenzek et al., Facebook, 2019) was designed to extract **high-quality monolingual** pre-training data, and is especially useful for **low-resource languages**:

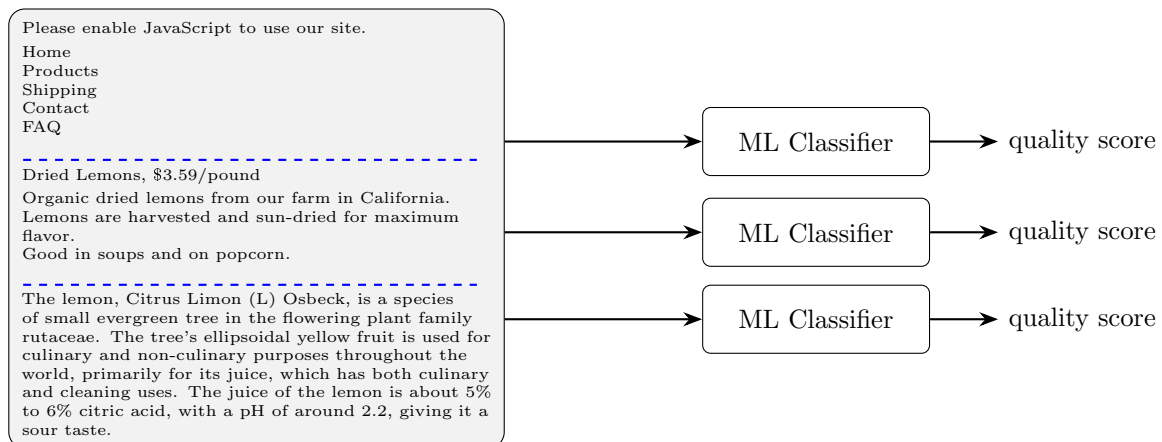
- BookCorpus and (large) Wikipedia are essentially English-only.
- Wikipedia in other languages is **too small**: Hindi has only $\sim 1.2\text{M}$ pages and Tamil $\sim 0.5\text{M}$, versus $\sim 59\text{M}$ for English.
- So we need Common Crawl dumps, often **years** of them. This is compute-intensive and needs massive parallelisation and carefully written code.



3.7.2 How CCNet works

1. **Paragraph-level deduplication** using hashing (normalised paragraphs are hashed; repeated paragraphs like navigation menus and cookie notices vanish).
2. **Language identification** (fastText).
3. **Targeted quality filtering**: score text with a language model pre-trained on a targeted domain (e.g. Wikipedia), and **remove low-quality pages based on a threshold**.

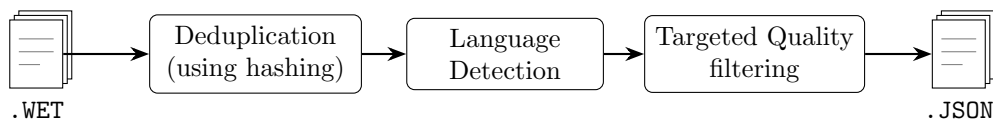
The slide's picture of the quality step: divide each webpage into three portions, **head**, **middle** and **tail**, and get a quality score for each with the classifier:



Here the head (JavaScript notice + menu) would score poorly, the middle (a product blurb) moderately, and the tail (encyclopaedic text) well.

Careful — what “head/middle/tail” means in the CCNet paper

In the original CCNet paper, *head*, *middle*, *tail* are **buckets of documents**, not parts of one page. For each language, a 5-gram KenLM trained on that language’s Wikipedia computes the perplexity of every (deduplicated) document. Documents are then split into three equal-size groups by perplexity tertile: the lowest-perplexity third is *head* (most Wikipedia-like), then *middle*, then *tail*. Many later works (e.g. RedPajama) keep only head+middle. The slide’s “split the page” picture gives the same intuition (score text against a reference LM, drop what scores badly). If an exam question is phrased as in the slides, answer as in the slides.

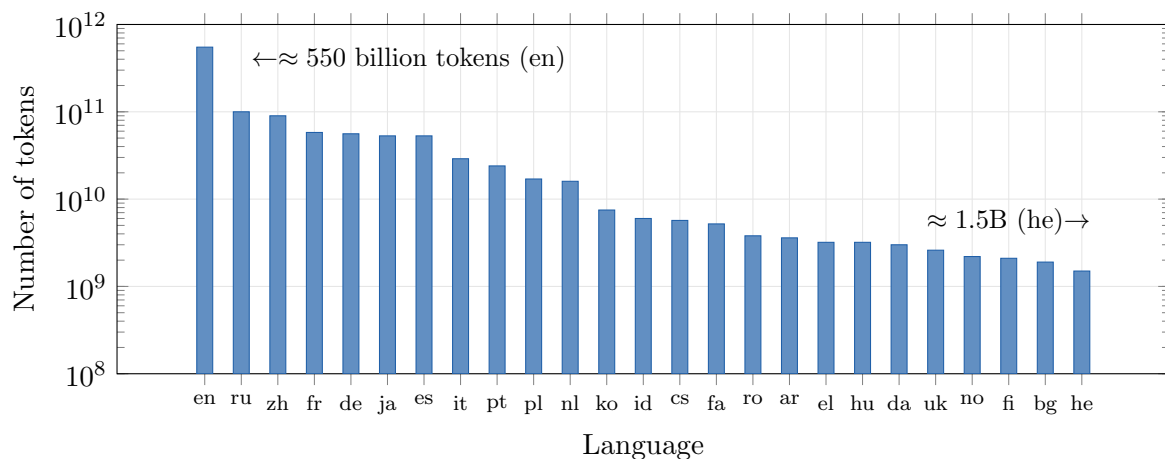


- It satisfies **three** requirements: dedup, LID and quality. Toxicity and PII are *not* handled.
- It extracted documents for **130 languages** (those with at least 1000 documents).

Intuition — why dedup *before* LID?

Dedup at paragraph level first removes boilerplate (menus, footers, cookie banners) that is repeated across millions of pages. That (i) shrinks the data cheaply, and (ii) makes LID more accurate, because a French article wrapped in an English site template would otherwise have its language signal diluted by the template.

3.7.3 CCNet output: tokens per language

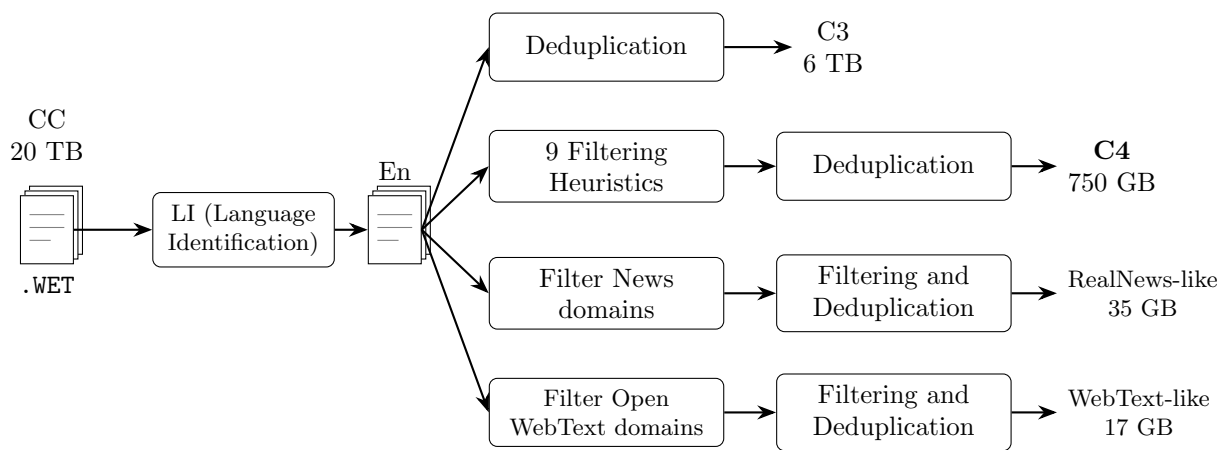


Number of tokens (SentencePiece) per language after deduplication, Feb 2019 CC snapshot. Bar heights are approximate readings of the slide figure.

- English dominates ($\approx 550\text{B}$ tokens) and Hebrew is at $\approx 1.5\text{B}$, about a $370\times$ gap on a *log* axis. The web is extremely imbalanced across languages.
- Recall that Tamil has 0.5M Wikipedia pages; CCNet on the *single* Feb 2019 snapshot yields **0.9M documents** for Tamil (there may be overlap with Wikipedia). One snapshot already beats Wikipedia, and years of snapshots multiply that.

3.8 The Colossal Clean Crawled Corpus (C4) pipeline

C4 (the slide title expands it as “Colossal Clean Common Crawl”), built for T5, is an **extension of CCNet with additional filtering heuristics**. By filtering on **domains** one can also get diverse sub-corpora (news, stories, web text).



- **C3**: English + dedup only, 6 TB. Big but dirty.
- **C4**: English + heuristics + dedup, **750 GB**. Released publicly.
- **RealNews-like** (35 GB) and **WebText-like** (17 GB): domain-filtered variants that emulate RealNews and OpenAI’s WebText.

Note the ratio: heuristics cut 6 TB to 750 GB, keeping only $\approx 12.5\%$. This shows how dirty raw web text is.

3.8.1 The C4 heuristics (as described in the T5 paper)

1. Keep only lines ending in a **terminal punctuation mark** (period, exclamation mark, question mark, end quote).
2. Discard pages with fewer than **5 sentences**; keep only lines with at least **3 words**.
3. Remove any page containing a word from the “List of Dirty, Naughty, Obscene or Otherwise Bad Words”.
4. Remove any line containing the word “**Javascript**” (warnings like “please enable JavaScript”).
5. Remove any page containing “**lorem ipsum**” (placeholder text).
6. Remove any page containing a **curly bracket** { (it is usually code).
7. Remove citation markers like [1], [citation needed], [edit].
8. Remove lines with boilerplate policy text (“terms of use”, “privacy policy”, “cookie policy”, “uses cookies”).
9. Deduplicate: discard all but one of any **three-sentence span** occurring more than once.

Plus: `langdetect` with $P(\text{English}) \geq 0.99$. (Different sources group these into “9” slightly differently. Remember the ideas, not the numbering.)

3.8.2 Worked illustration: cleaning three “lemon” pages

CC’s “Web Extracted Text” contains a lot of gibberish, boilerplate (menus), offensive language, placeholder text and source code. We want a clean corpus suitable for NLP. Here are three raw WET pages (source: Colin Raffel’s slides), followed by what each rule removes (~~struck-through~~ text is removed).

Stage 0: raw text

Menu Lemon Introduction The lemon, Citrus Limon (L) Osbeck, is a species of small evergreen tree in the flowering plant family rutaceae. The tree's ellipsoidal yellow fruit is used for culinary and non-culinary purposes throughout the world, primarily for its juice, which has both culinary and cleaning uses. The juice of the lemon is about 5% to 6% citric acid, with a ph of around 2.2, giving it a sour taste. Article The origin of the lemon is unknown, though lemons are thought to have first grown in Assam (a region in northeast India), northern Burma or China. A genomic study of the lemon indicated it was a hybrid between bitter orange (sour orange) and citron.	Please enable JavaScript to use our site. Home Products Shipping Contact FAQ Dried Lemons, \$3.59/pound Organic dried lemons from our farm in California. Lemons are harvested and sun-dried for maximum flavor. Good in soups and on popcorn. The lemon, Citrus Limon (L) Osbeck, is a species of small evergreen tree in the flowering plant family rutaceae. The tree's ellipsoidal yellow fruit is used for culinary and non-culinary purposes throughout the world, primarily for its juice, which has both culinary and cleaning uses. The juice of the lemon is about 5% to 6% citric acid, with a ph of around 2.2, giving it a sour taste	Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur in tempus quam. In mollis et ante at consectetur. Aliquam erat volutpat. Donec at lacinia est. Duis semper, magna tempor interdum suscipit, ante elit molestie urna, eget efficitur risus nunc ac elit. Fusce quis blandit lectus. Mauris at mauris a turpis tristique lacinia at nec ante. ... <pre>function Ball(r){ this.radius = r; this.area = pi * r ** 2; this.show = function() { drawCircle(r); } }</pre>
--	---	--

Rule A: remove any page containing “lorem ipsum” or a curly bracket. Page 3 is deleted *entirely* (it has both placeholder text and JavaScript code).

Rule B: remove lines without terminal punctuation. From page 1: ~~Menu~~, ~~Lemon~~, ~~Introduction~~, ~~Article~~. From page 2: ~~Home~~, ~~Products~~, ~~Shipping~~, ~~Contact~~, ~~FAQ~~, ~~Dried Lemons, \$3.59/pound~~, plus the last paragraph (“...giving it a sour taste” has *no* final period!). The line “Please enable JavaScript to use our site.” goes via the “Javascript” rule.

Rule C: deduplicate the text. The encyclopaedic paragraph “The lemon, Citrus Limon (L) Osbeck ...” appears in both page 1 and page 2, so the second copy is removed.

Rule D: remove bad words using a manual list. Any page containing a listed word is removed *entirely* (none here).

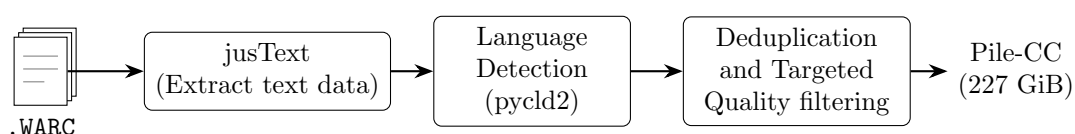
What survives: page 1’s two encyclopaedic paragraphs (origin and description of the lemon) and page 2’s three product-description sentences (“Organic dried lemons ...”, “Lemons are harvested ...”, “Good in soups and on popcorn.”). Clean, natural, punctuated English.

Intuition — why such blunt rules work

Each rule is a cheap *proxy* for “is this natural prose?”. Humans end sentences with punctuation, but menus and titles don’t. Code has curly brackets and prose rarely does. Placeholder text has “lorem ipsum”. None of these is perfect (C4’s curly-bracket rule deletes all pages with code, and the bad-word rule is known to remove non-toxic text by and about marginalised groups), but at the scale of billions of pages, cheap high-recall rules are what is affordable.

3.9 Variations on the theme: Pile-CC

In general, all pre-processing pipelines follow the CCNet pipeline, with modifications to its components. For example, **Pile-CC** (the Common Crawl part of The Pile) uses **WARC** files (raw HTML) instead of WET, and extracts text with **jusText**:



- **jusText** classifies each HTML block as boilerplate or content, using link density, stop-word density and block length, then keeps the content blocks. This gives much cleaner text than WET.
- **pycld2** (Compact Language Detector 2) for LID.
- Uses **fuzzy deduplication** (MinHash LSH) instead of the *exact* deduplication used in C4. Quality filtering uses a fastText classifier trained to separate OpenWebText2 (positive) from raw CC (negative).

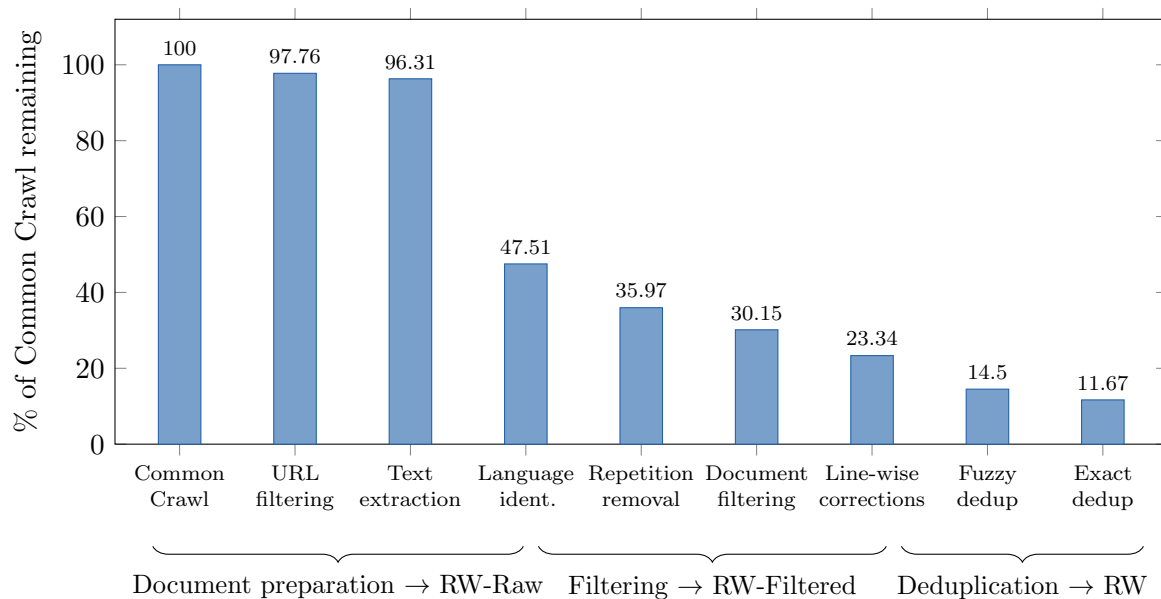
Order of dedup. Significant duplicates in CC are *exact* duplicates, so the slide suggests one can use fuzzy dedup first followed by exact dedup. That is what RefinedWeb does: document-level MinHash first removes near-duplicate *documents*, then exact *substring* dedup (suffix array) removes repeated spans inside the documents that remain. The two passes work at different granularities, so they complement each other.

3.10 RefinedWeb (RW): the stringent web-only pipeline

RefinedWeb (TII, for the Falcon LLM) set out to show that **web data alone**, processed carefully enough, can match or beat curated corpora.



Stages (percentages are of the original CC tokens that *remain* after each step):



Step	Remaining (% of CC)	Removed (points of CC)	Removed, relative (% of step input)
URL filtering	97.76	2.24	2.24%
Text extraction	96.31	1.45	1.49%
Language identification	47.51	48.80	50.66%
Repetition removal	35.97	11.54	24.28%
Document-wise filtering	30.15	5.82	16.19%
Line-wise corrections	23.34	6.81	22.59%
Fuzzy dedup (MinHash)	14.50	8.84	37.88%
Exact dedup (substring)	11.67	2.83	19.52% [†]

[†]Computed as 2.83/14.50. The slide’s Sankey figure prints 18.47% for this step, probably measured on a slightly different basis. If asked to derive it from the “remaining” column, use 19.5%.

How the slide’s summary numbers fall out of this table (they are *percentage points of the original CC*):

- **≈50% of CC removed after language identification:** 96.31 → 47.51 (48.8 points; only English is kept).
- **≈24% removed by quality filtering:** RW-Raw 47.51 → RW-Filtered 23.34, i.e. 24.17 points.
- **≈12% removed by deduplication:** 23.34 → 11.67, i.e. 11.67 points (half of RW-Filtered!).
- **In total, almost 90% removed:** $100 - 11.67 = 88.33\%$ of a single CC snapshot is discarded.

Beyond the slides — what the RefinedWeb filters are

URL filtering: a blocklist of ~4.6M adult/fraud domains plus URL keyword scoring. **Trafilatura:** HTML→text extraction. **LID:** fastText, keep English with score ≥ 0.65 . **Repetition removal & document-wise filtering:** Gopher-style heuristics (e.g. reject documents with too many repeated lines or n -grams, word counts outside 50–100,000, mean word length outside 3–10 characters, symbol-to-word ratio > 0.1 , too few stop words). **Line-wise corrections:** delete lines like “share on Facebook”, counters and navigation. **Fuzzy dedup:** MinHash on 5-grams, 9000 hashes/doc in 20 buckets. **Exact dedup:** suffix-array removal of repeated spans of ≥ 50 tokens.

Then the lecture asks: what is the impact of having duplicate content? (Answered in Section 3.14.)

3.11 Comparison of pipelines (from the RefinedWeb paper)

Dataset	Size	Availability	Web	CC processing	Deduplication
MASSIVE WEB DATASETS					
C4	~360GT	Public	100%	Rules + NSFW words blocklist	Exact: spans of 3 sentences
OSCAR-21.09	~370GT	Public	100%	Built at the line level	Exact: per line (~55% removed)
OSCAR-22.01	~283GT	Public	100%	Line-level rules + optional rules & NSFW URL blocklist	Exact: per line (optional)
CURATED DATASETS					
GPT-3	300GT	Private	60%	Content filter trained on known high-quality sources	Fuzzy: MinHash (~10% removed)
The Pile	~340GT	Public	18%	jusText for extraction, content filter trained on curated data	Fuzzy: MinHash (~26% removed)
PaLM	780GT	Private	27%	Filter trained on HQ data	Unknown
OURS					
RefinedWeb	~5000GT	Public (600GT)	100%	trafilatura for text extraction, document- and line-level rules, NSFW URL blocklist	Exact & fuzzy: exact substring + MinHash (~50% removed)

GT = giga-tokens = billions of tokens.

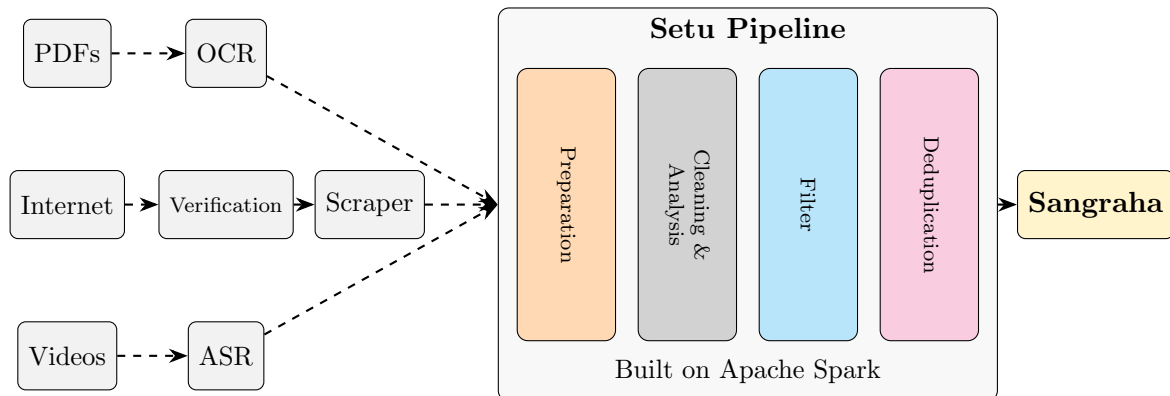
Takeaways: (i) curated datasets use only 18–60% web, while RefinedWeb is 100% web and ~5T tokens; (ii) the stronger pipelines combine *fuzzy* and *exact* dedup; (iii) many important datasets

(GPT-3, PaLM) are private.

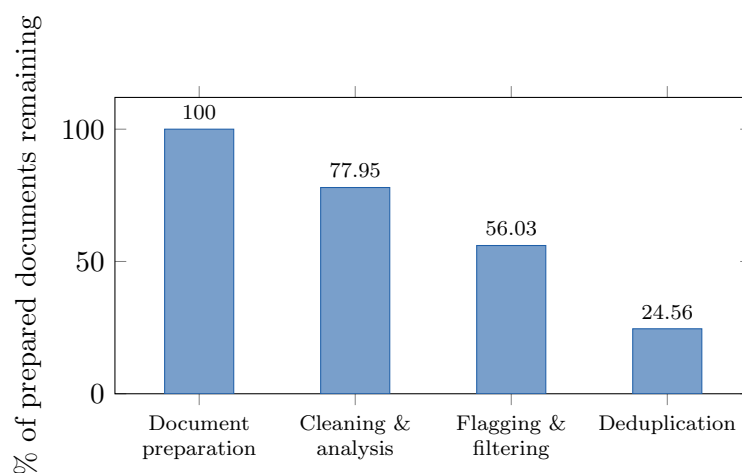
Beyond text on the web: other modalities

So far every pipeline takes in text. Why not use **audio** (via ASR, automatic speech recognition) or **scanned documents** (via OCR)? These modalities are **especially helpful for low-resource languages**. Many Indian languages have far more spoken content (YouTube, radio) and printed books than web text.

3.12 The SETU pipeline (Sangraha, AI4Bharat)



Name of the dataset	Sangraha Verified
Data sources	Web (curated and <i>manually verified</i>), Videos (ASR), Digitised PDFs (OCR)
Scraping	Trafilatura
Language detection	IndicLID, CLD3, NLLB (LID model)
Deduplication	Fuzzy (MinHash)
Infrastructure	Apache Spark (distributed)



Removed in *points*: cleaning 22.05, flagging/filtering 21.91, dedup 31.47. Only $\approx 24.6\%$ survives. Even after *manual verification* of websites, dedup is the single biggest cut. Indian-language web content is heavily copied and re-posted across news and aggregator sites.

3.13 Perfect deduplication is difficult

- A single **61-word sentence was repeated 60 000 times in C4** (Lee et al., 2021).
- This is not specific to C4. Other datasets (RealNews, LM1B, Wiki40B) contain duplicates too, *including* overlap between their train and validation splits:

	% train tokens with		% valid with
	dup in train	dup in valid	dup in train
C4	7.18%	0.75%	1.38%
RealNews	19.4%	2.61%	3.37%
LM1B	0.76%	0.016%	0.019%
Wiki40B	2.76%	0.52%	0.67%

- And The Pile: “We found the Pile was particularly full of duplicate documents, and advise future researchers using the Pile to perform additional de-duplication processing” (OPT paper).
- Footnote on the slide: the **Pythia** suite found that deduplicating The Pile had **limited impact** on zero-shot performance. The evidence is not one-sided.

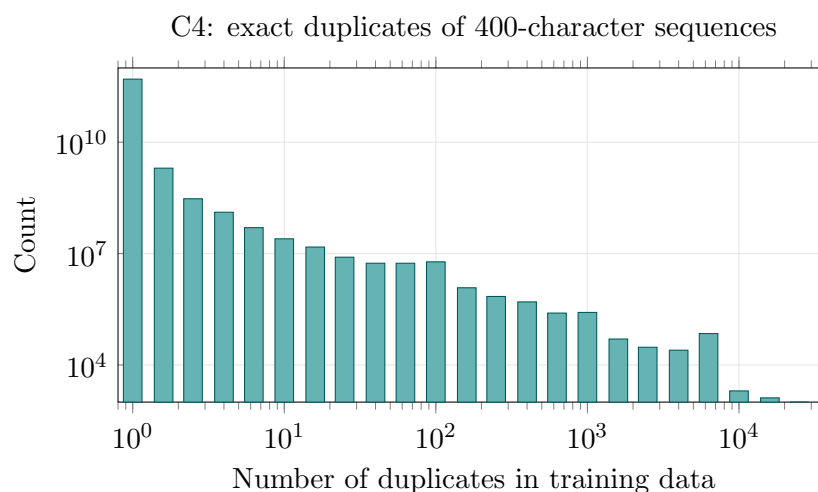
Intuition — reading the columns

“% valid with dup in train” is **train–test contamination**. For RealNews, 3.37% of validation examples have near-copies in training, so validation perplexity *overstates* generalisation, because the model is partly tested on things it memorised.

3.14 Why is deduplication important?

Duplication:

1. allows the model to **memorise** the training data;
2. gives a **wrong picture of test performance** (contamination, as above);
3. makes the model **susceptible to privacy risks**: memorised PII can be extracted by prompting.



Approximate redraw of the slide figure (Lee et al. 2021). Both axes are logarithmic.

- **Millions of sequences were repeated at least 10 times** in C4.
- A sequence present **10 times** in the training data is, **on average**, generated **1000× more often** than a sequence present only once (Kandpal et al., 2022).
- Scale makes it worse: **for a 1B-parameter model, a hundred duplicates are harmful; at 175B, even a few duplicates could have a disproportionate effect.** Larger models memorise faster.

Derivation — memorisation is super-linear in the duplicate count

Suppose the generation rate g of a sequence scales as a power of its duplicate count k : $g(k) \propto k^\gamma$. The slide's data point says $g(10)/g(1) = 1000$, so

$$10^\gamma = 10^3 \Rightarrow \gamma = 3.$$

Under this (illustrative) power-law reading, doubling the copies of a sequence makes it $2^3 = 8\times$ more likely to be regurgitated. That is why removing even a modest number of heavily repeated sequences disproportionately reduces memorisation.

3.15 All pipelines are leaky!

Four observations from the literature:

1. **Open Pre-trained Transformer (OPT):** *“We found the Pile was particularly full of duplicate documents, and advise future researchers using the Pile to perform additional de-duplication processing.”* Even a well-known curated dataset leaks duplicates.
2. **Learning from Repeated Data** (Hernandez et al., Anthropic): the performance of an **800M**-parameter model can be degraded to that of a **2× smaller (400M)** model by **repeating 0.1% of the data 100 times**, even though the other 90% of the training tokens stay unique.
Arithmetic check: 0.1% of the unique data $\times$ 100 repeats $\approx$ 10% of all training tokens. So 90% unique + 10% repeated, where the repeated part is just 0.1% of the underlying data.
3. **Insights from training Gopher:** *“We do not attempt to filter out low quality documents by training a classifier based on a ‘gold’ set of text, such as English Wikipedia or pages linked from Reddit, as this could inadvertently bias towards a certain demographic or erase certain dialects or sociolects from representation.”*
4. **A Pretrainer’s Guide:** *“A ‘high-quality’ Reddit post does not look like a ‘high-quality’ academic paper; and even with academic papers, quality measured by peer review has high variance.”*

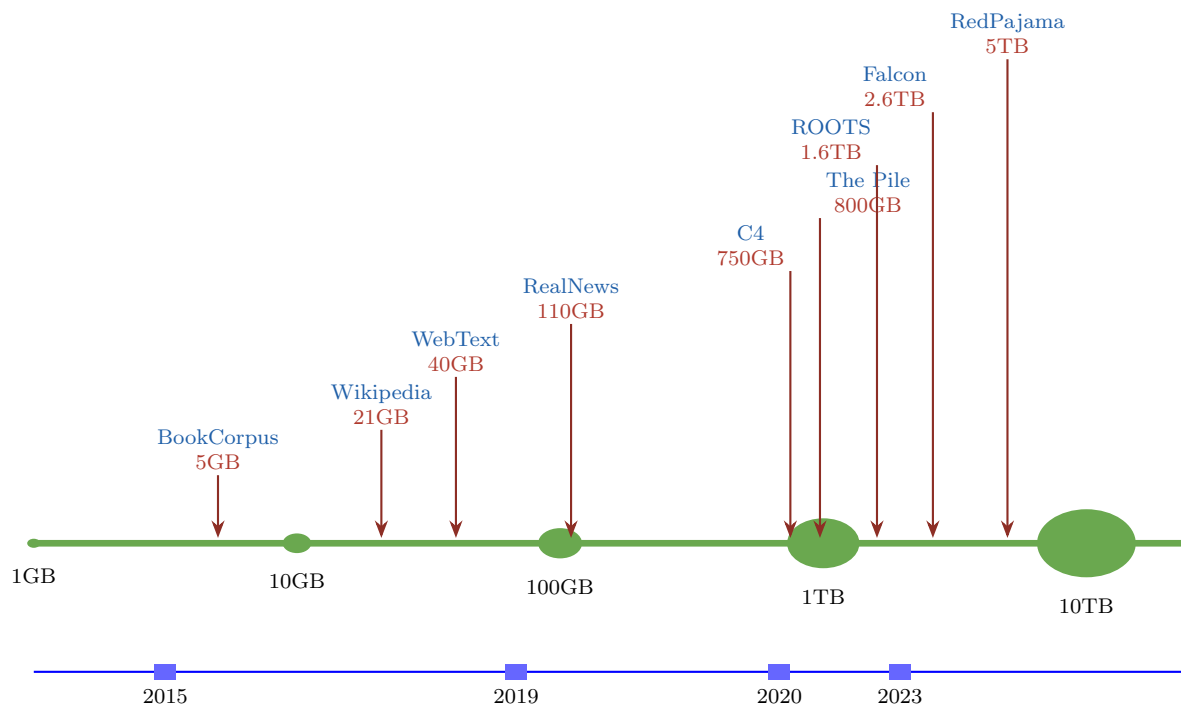
Key idea — “quality” is not a single number

Classifier-based quality filtering replaces the vague question “is this good text?” with “does this look like my reference set?”. Whatever the reference under-represents (dialects, informal registers, code-mixed Hinglish, minority communities) gets filtered out, which hurts *diversity* and *fairness*. Heuristics avoid some of this bias but are crude. There is no leak-proof pipeline, only trade-offs.

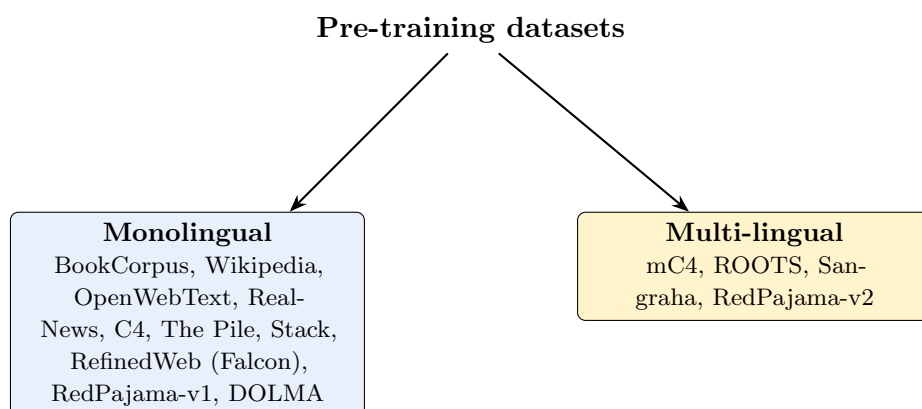
4 Module 3: Pre-training datasets

4.1 Availability problems

- Some publicly released pre-training datasets are **no longer available** (e.g. **BookCorpus**, withdrawn over copyright concerns).
- Some datasets are **private** (e.g. OpenAI's **WebText**).
- So researchers **build their own versions** by scraping the web and following the steps in the original paper, e.g. **OpenWebText** (a replication of WebText: Reddit outbound links with ≥ 3 karma) and **BookCorpus2** (in the Pile).



4.2 A taxonomy of pre-training datasets



The lecture focuses on: **C4, The Pile, Stack, RefinedWeb (Falcon), RedPajama-v1/v2, DOLMA, mC4, ROOTS, Sangraha.**

4.3 How to compare dataset sizes

1. By **(uncompressed) storage**, in GB/TB. This is easy to measure but depends on encoding (a Tamil character takes 3 bytes in UTF-8, an English one takes 1).
2. By **number of tokens**, which relates directly to the scaling law's D and is therefore more meaningful for training. It depends on the tokenizer.

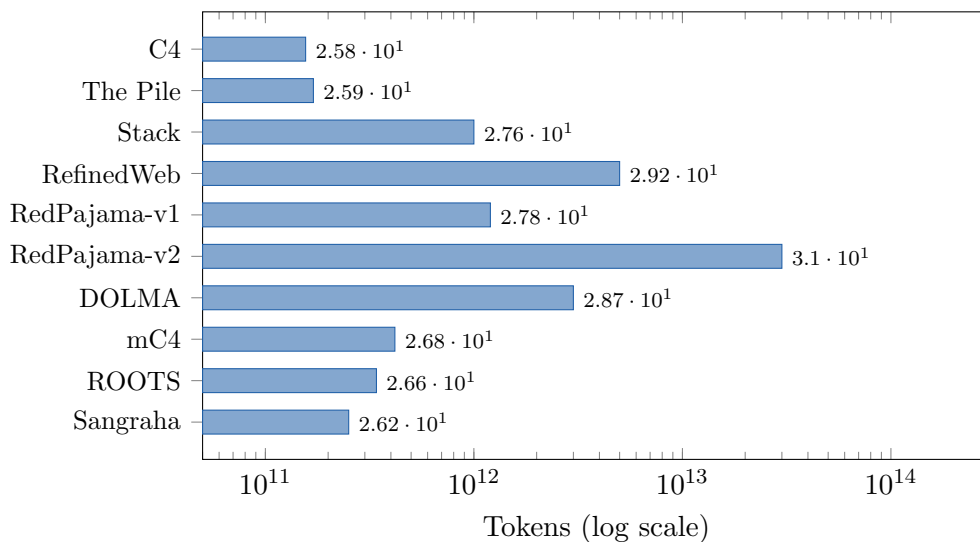
Datasets such as **The Pile** and **DOLMA** emphasise **diversity**. Datasets such as **RefinedWeb** suggest one can get good performance with **web-only** data.

Beyond the slides — bytes per token across scripts

In UTF-8, Latin letters take 1 byte, Devanagari/Tamil 3 bytes, most CJK characters 3 bytes. An English-centric tokenizer also splits Indic words into many more tokens. So “1 GB of Hindi” and “1 GB of English” can differ several-fold in both *content* and *token count*. This is one more reason the lecture prefers counting tokens.

4.4 The datasets at a glance

Dataset	Languages	# of tokens	Curated?	Diversity
C4	English	~156 Billion	No	Webpage
The Pile	English	~170 Billion	Yes	22 sources
Stack	Code	> 1 Trillion	Yes	380 programming languages
RefinedWeb (Falcon)	English	5 Trillion (600B public)	No	Webpage
RedPajama-v1/v2	English/Multi	1.2 / 30 Trillion	Yes	Webpage, Books, ArXiv, Wiki, StackExchange
DOLMA	English	3 Trillion	Yes	Webpage, Books, Wiki, The Stack, STEM
mC4	Multi	~418 Billion	No	Webpage
ROOTS	Multi	~341 Billion	Yes	Natural and programming languages
Sangraha	Multi	251 Billion	Yes	Web, videos, digitised PDF, synthetic



Notes on each dataset

C4

English CC after the heuristics of Module 2. Not curated: a single source type (webpages).

The Pile (EleutherAI, 2020)

22 hand-picked sources (academic, code, legal, books, dialogue). *Curated for diversity.*

The Stack (BigCode)

Permissively licensed source code in 380 programming languages, >1T tokens. Used for code models (StarCoder).

RefinedWeb

Web-only, stringent filtering and dedup (Module 2). 5T tokens in total, 600B public.

RedPajama-v1

An *open reproduction of the LLaMA training mix* (1.2T tokens). **v2**: 30T raw tokens over 5 languages, shipped with quality signals so users can filter themselves.

DOLMA (AI2, 2024)

3T tokens, open, used for OLMo. Web + code + Reddit + STEM papers + books + encyclopaedic.

mC4

The multilingual C4 (101 languages), built for mT5.

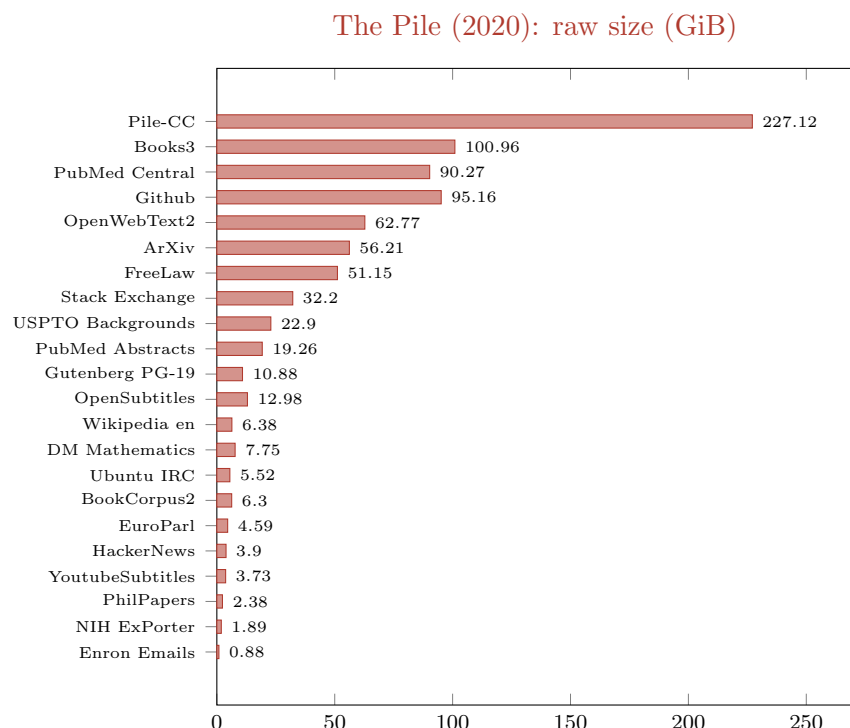
ROOTS (BigScience)

46 natural and 13 programming languages, for BLOOM.

Sangraha (AI4Bharat)

22 Indian languages; web (manually verified), ASR on videos, OCR on PDFs, plus synthetic (translated) data.

4.5 Composite datasets



RedPajama-V1 (2022/23)

Dataset	Token count
CommonCrawl	878 B
C4	175 B
GitHub	59 B
Books	26 B
ArXiv	28 B
Wikipedia	24 B
StackExchange	20 B
Total	1.2 T

DOLMA (2024)

Source	Doc type	Llama tokens (B)
Common Crawl	web pages	2,281
The Stack	code	411
C4	web pages	198
Reddit	social media	89
PeS2o	STEM papers	70
Project Gutenberg	books	6.0
Wikipedia, Wikibooks	encyclopedic	4.3
Total		3,059

The Pile total: 825.18 GiB (~ 170 B tokens). RedPajama-V1: after extensive deduplication the tokens reduce to 670B; the result is called **SlimPajama**.

Observe the increase in token counts each year: Pile (~ 0.17 T, 2020) $\rightarrow$ RedPajama-V1 (1.2T, 2023) $\rightarrow$ DOLMA (3.06T, 2024). That is roughly $18\times$ in four years, and *web is still the majority* in every one of them.

Exam tip

Quick sanity sums: RedPajama $878 + 175 + 59 + 26 + 28 + 24 + 20 = 1210\text{B} \approx 1.2\text{T}$. DOLMA $2281 + 411 + 198 + 89 + 70 + 6 + 4.3 = 3059.3\text{B}$. SlimPajama keeps $670/1210 \approx 55\%$, i.e. dedup removed $\approx 45\%$ of RedPajama-V1. This is the lecture's "perfect dedup is difficult / pipelines are leaky" point in numbers.

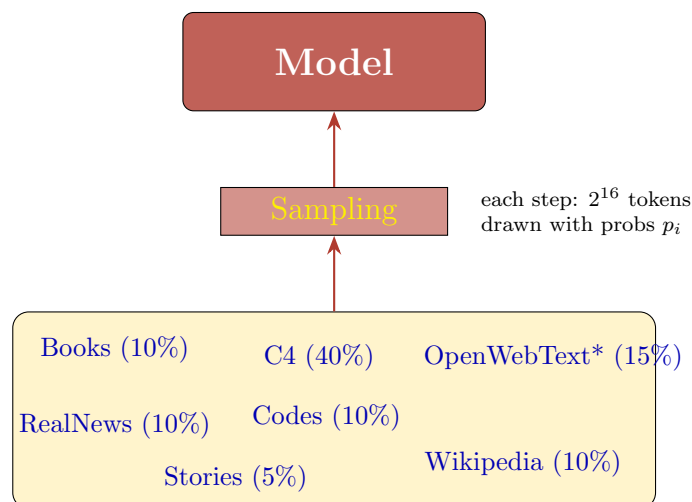
4.6 Data diversity and data mixing

Using **high-quality and diverse** datasets is crucial for optimal downstream performance. *How* do we combine sources during training?

Setup from the slide. Train for **15 million steps** with 2^{16} **tokens per step**:

$$T = 15 \times 10^6 \times 2^{16} = 15 \times 10^6 \times 65\,536 \approx 9.83 \times 10^{11} \approx \mathbf{1 \text{ trillion tokens.}}$$

For each step, the 2^{16} tokens are drawn by **sampling from all available sources according to a pre-defined distribution**:



*OpenWebText is an open version of WebText.

Derivation — tokens per source, epochs, and why proportions $\neq$ sizes

Let source i have D_i unique tokens and mixture weight p_i ($\sum_i p_i = 1$). With total training budget T tokens:

$$\begin{aligned} \text{tokens drawn from } i : \quad T_i &= p_i T && [\text{tokens}] \\ \text{expected tokens from } i \text{ per step} : \quad p_i \cdot 2^{16} &&& [\text{tokens/step}] \\ \text{epochs over source } i : \quad E_i &= \frac{p_i T}{D_i} && [\text{dimensionless}] \end{aligned}$$

If we sampled *proportionally to size*, $p_i = D_i / \sum_j D_j$, every source would get the same number of epochs $E = T / \sum_j D_j$. Real mixtures **up-weight** small, high-quality sources (Wikipedia, books) so they get $E_i > 1$, and **down-weight** huge noisy ones (CC), so $E_i < 1$: part of CC is never seen.

Example (slide mix): C4 at 40% of 2^{16} is 26 214 tokens per step on average, and $0.4 \times 9.83 \times 10^{11} \approx 393\text{B}$ tokens over training, i.e. ≈ 2.5 epochs of a 156B-token C4.

Intuition — the tension with Module 1

Up-weighting Wikipedia means *repeating* it, which is exactly what Module 1 warned against. Mixtures therefore keep repetition modest (≈ 2 – 3 epochs at most for LLaMA’s Wikipedia/Books). The designer trades *quality* (show more good text) against *overfitting* (don’t repeat too much).

Note, however, that **RefinedWeb** demonstrates that sourcing large-scale, high-quality data from the web alone can be sufficient for better performance.

4.6.1 Real mixtures: GPT-3, LLaMA, PaLM**GPT-3** (300B training tokens)

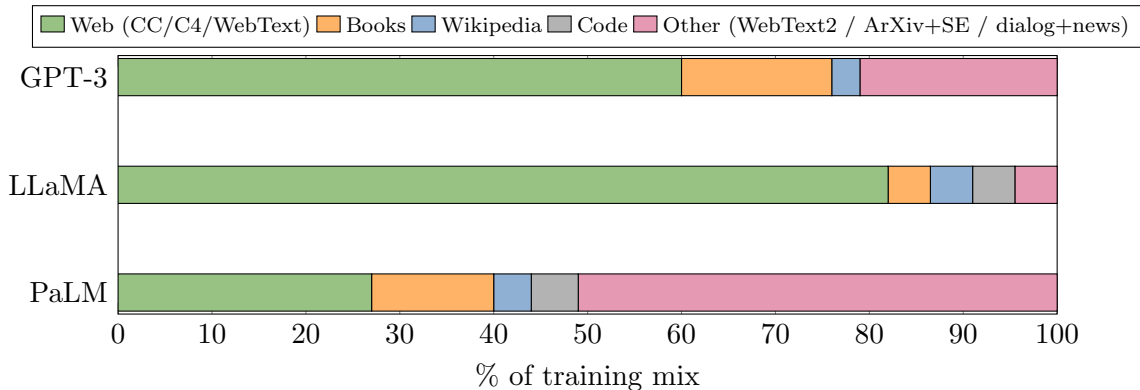
Dataset	Quantity (tokens)	Weight in mix
Common Crawl (filtered)	410 B	60%
WebText2	19 B	22%
Books1	12 B	8%
Books2	55 B	8%
Wikipedia	3 B	3%

LLaMA ($\sim 1.4\text{T}$ training tokens)

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

PaLM (total 780B tokens)

Data source	Proportion
Social media conversations (multilingual)	50%
Filtered webpages (multilingual)	27%
Books (English)	13%
GitHub (code)	5%
Wikipedia (multilingual)	4%
News (English)	1%



Extra figure. GPT-3’s “Other” is WebText2 (22%, curated web); LLaMA’s is ArXiv + StackExchange; PaLM’s is conversations + news. GPT-3’s weights as published sum to 101% due to rounding.

Key idea — reading the epochs column

The epochs column confirms the up-weighting logic. LLaMA shows Wikipedia 2.45 epochs and Books 2.23 (small, high quality, repeated), CommonCrawl 1.10, and GitHub **0.64**: less than one pass, since code was deliberately down-sampled. GPT-3 is similar: WebText2 got $0.22 \times 300/19 \approx 3.5$ epochs while Common Crawl got only $0.6 \times 300/410 \approx 0.44$.

4.7 Represented domains across models

Model	Represented domains (%)									Filters		Data	
	Wiki	Web	Books	Dialog	Code	Acad	Pile	C4	M-L	Tox	Qual	Pub	Year
BERT	76		24				×	×			H	Part	2018
GPT-2		100					×	×			H	Part	2019
RoBERTa	7	90	3				×	✓			H	Part	2019
XLNet	8	89	3				×	✓			H	Part	2019
T5	<1	99					×	✓		H	H	✓	2019
GPT-3	3	82	16				×	✓	7%		C	×	2021
GPT-J/Neo	1.5	38	15	4.5	13	28	✓	Part			C	✓	2020
GLaM	6	46	20	28			×	✓			C	×	2021
LaMDA	13	24		50	13		✓	✓	10%	C	C	×	2021
AlphaCode					100		×	×			H	×	2021
CodeGen	1	24	10	3	40	22	✓	Part			H	Part	2020
Chinchilla	1	65	10		4		✓	✓		H	C	×	2021
Minerva	<1	1.5	<1	2.5	<1	95	✓	✓	<1%		C	×	2022
BLOOM	5	60	10	5	10	10	✓	✓	71%	H	C	Part	2021
PaLM	4	28	13	50	5		×	✓	22%		C	×	2021
Galactica	1	7	1		7	84	✓	Part			H	Part	2022
LLaMA	4.5	82	4.5	2	4.5	2.5	Part	✓	4%		C	Part	2020

M-L: multilingual share. Filters: H = heuristics, C = classifiers. Pub: data publicly available. Source: *A Pretrainer’s Guide to Training Data*. (Years as printed on the slide.)

What the table shows:

- **Almost all LLMs predominantly use web data** (from The Pile, C4, CC, or custom scraping).
- **Wiki and Books** are the next most common sources.
- **AlphaCode** uses **100% code**; **Minerva** (95% academic/math) and **Galactica** (84% academic) are domain-specialised.
- **LaMDA and PaLM** give more weight to **conversation (dialog) data** (50% each). They were designed for dialogue.
- Some models were trained on **multilingual** data: BLOOM (71%), PaLM (22%), LaMDA (10%), GPT-3 (7%), LLaMA (4%).
- **Filters**: newer/bigger models tend to use *classifier-based* quality filters (C), while older ones used heuristics (H). Toxicity filtering is rare.
- **Data publication**: most frontier models' data is not (fully) public.

Intuition — you are what you eat

The mix tells you the model's personality. Mostly-web gives a generalist. Heavy dialog gives a chatty model (LaMDA). Heavy academic text gives a math/science specialist (Minerva, Galactica). All code gives a coder (AlphaCode). *Data mixture is a design decision as important as architecture.*

4.8 Some questions to ponder (with discussion)

1. **Are we reaching the limit on the size of the data?** Possibly, for *high-quality human-written* text. Deduplicated, filtered English web text is on the order of tens of trillions of tokens, and frontier training runs already consume a sizeable fraction of that. The scaling law's data term then becomes a hard bottleneck. Responses: multi-epoch training (with care), more languages and modalities (ASR/OCR), and synthetic data.
2. **Could we use LLMs like GPT-4 to generate synthetic data?** Yes, and it is already done (e.g. "Textbooks Are All You Need"/phi-1 used GPT-generated textbook-style data; Sangraha includes synthetic translated data). Risks: errors and biases of the teacher are inherited; diversity can collapse; training recursively on model outputs can degrade distributions ("model collapse"). Synthetic data works best when targeted, filtered and mixed with real data.
3. **Would small (30B–100B tokens) but high-quality datasets ever be helpful?** Yes, especially for small models and specialised skills: phi-1 (1.3B params, ~7B tokens) achieved strong coding results. High quality raises sample efficiency, but by the scaling law it cannot remove the data term for large models. Quality shifts the curve; scale moves along it.
4. **Is mixing code (non-ambiguous) and text (ambiguous) helpful or harmful?** Empirically, adding a moderate share of code tends to *help* reasoning and structured-output tasks without hurting natural-language ability much. Too much code dilutes natural-language data, though. It is a mixture-weight question again.

4.9 References (from the slides)

- Stanford CS324 (Winter 2022), lecture on data:
<https://stanford-cs324.github.io/winter2022/lectures/data/>

- Wenzek et al., *CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data*.
- Raffel et al., *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* (T5, C4).
- Gao et al., *The Pile: An 800GB Dataset of Diverse Text for Language Modeling*.
- Penedo et al., *The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only*.
- AI4Bharat, *IndicLLMSuite: A Blueprint for Creating Pre-training and Fine-Tuning Datasets for Indian Languages*.
- Longpre et al., *A Pretrainer's Guide to Training Data: Measuring the Effects of Data Age, Domain Coverage, Quality, & Toxicity*.
- Also cited in the lecture: Kaplan et al. (Scaling Laws); Lee et al. (Deduplicating Training Data Makes Language Models Better); Kandpal et al. (Deduplicating training data mitigates privacy risks); Hernandez et al. (Scaling Laws and Interpretability of Learning from Repeated Data); Rae et al. (Gopher); Zhang et al. (OPT); Touvron et al. (LLaMA); Gunasekar et al. (Textbooks Are All You Need).

5 One-page summary (revision sheet)

Module 1 — Why scale data?

- T5: repeating a small corpus for many epochs $\Rightarrow$ overfitting. Prefer fresh data.
- Kaplan scaling law: $L(N, D) = [(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$, with $N_c = 8.8 \times 10^{13}$, $D_c = 5.4 \times 10^{13}$, $\alpha_N = 0.076$, $\alpha_D = 0.095$, $\alpha_N/\alpha_D = 0.8$.
- Limits: $L(N, \infty) = (N_c/N)^{\alpha_N}$ (model-limited floor) and $L(\infty, D) = (D_c/D)^{\alpha_D}$ (data-limited).
- Slide example: $N = 10^8$, $D = 10^7$ gives train loss ≈ 0 and test loss 4.36. More D lowers test loss toward the floor 2.83.
- To keep the overfitting penalty fixed: $D \propto N^{0.8}$. Scale model *and* data.
- Three axes of data: **scale**, **quality**, **diversity**. Trends: diversity (Pile, RedPajama), quality (RefinedWeb, Textbooks), open data (LLaMA).

Module 2 — Sourcing & cleaning

- Web > 100 PB. Common Crawl: monthly snapshot of 3–5B pages in WARC (raw HTML, 86.77 TiB), WAT (metadata, 21.11 TiB), WET (text, 8.69 TiB).
- 5 mechanisms: LID (fastText/langdetect/pycld2/IndicLID); dedup (hashing, suffix array; MinHash/SimHash/LSH); toxicity (word lists, classifiers, Perspective API); quality (reference-LM perplexity, heuristics); PII (regex).
- fastText basic: WET $\rightarrow$ LID $\rightarrow$ line-level hash dedup (2 of 5 requirements).
- CCNet: WET $\rightarrow$ paragraph dedup $\rightarrow$ LID $\rightarrow$ LM quality filter (head/middle/tail) $\rightarrow$ JSON; 130 languages (3 of 5).
- C4: CCNet + heuristics (terminal punctuation, ≥ 5 sentences, ≥ 3 words/line, bad words, “javascript”, “lorem ipsum”, {, 3-sentence dedup). C3 is 6 TB and C4 is 750 GB.
- Pile-CC: WARC $\rightarrow$ jusText $\rightarrow$ pycld2 $\rightarrow$ fuzzy dedup + quality (227 GiB). RefinedWeb: WARC $\rightarrow$ Trafalatura+LID $\rightarrow$ quality $\rightarrow$ fuzzy+exact dedup, keeping 11.67% (almost 90% removed).
- SETU/Sangraha: web (verified) + OCR(PDF) + ASR(video); IndicLID/CLD3/NLLB; MinHash; Spark. 24.56% survives.
- Dedup matters: memorisation (10 copies give $1000\times$ more generation), inflated test scores, privacy. Pipelines are leaky; “quality” classifiers can bias.

Module 3 — Datasets & mixing

- Monolingual: BookCorpus, Wikipedia, OpenWebText, RealNews, C4, Pile, Stack, RefinedWeb, RedPajama-v1, DOLMA. Multilingual: mC4, ROOTS, Sangraha, RedPajama-v2.
- Size by storage or (better) tokens. C4 156B; Pile 170B; Stack >1T; RefinedWeb 5T (600B public); RedPajama 1.2T/30T; DOLMA 3T; mC4 418B; ROOTS 341B; Sangraha 251B.
- Mixing: sample each batch (2^{16} tokens) from sources with probabilities p_i . $T_i = p_i T$, epochs $E_i = p_i T / D_i$. Up-weight small high-quality sources, down-weight CC.
- GPT-3 (CC 60%, WebText2 22%, Books 8+8%, Wiki 3%); LLaMA (CC 67%, C4 15%, GitHub/Wiki/Books 4.5% each, ArXiv 2.5%, SE 2%); PaLM (50% conversations).
- Most LLMs are web-dominated; LaMDA/PaLM are dialog-heavy; AlphaCode is 100% code; Minerva and Galactica are academic-heavy.

Formula card

Scaling law	$L = [(N_c/N)^{0.8} + D_c/D]^{0.095}$
Data-limited doubling	$L(2D)/L(D) \rightarrow 2^{-0.095} = 0.936$
Overfit-free growth	$D \propto N^{\alpha_N/\alpha_D} = N^{0.8}$
Non-embedding params	$N \approx 12 n_{\text{layer}} d_{\text{model}}^2$
Perplexity	$\text{PPL} = \exp\left(-\frac{1}{T} \sum_t \ln q(w_t w_{<t})\right)$
Jaccard	$J(A, B) = A \cap B / A \cup B $
MinHash	$P[\min h(A) = \min h(B)] = J, \quad \text{Var}(\hat{J}) = J(1 - J)/k$
LSH banding	$P(\text{cand}) = 1 - (1 - s^r)^b, \quad \text{threshold} \approx (1/b)^{1/r}$
SimHash	$P[\text{bit agree}] = 1 - \theta/\pi$
Hash collisions	$\mathbb{E}[\#\text{collisions}] \approx n^2/2^{b+1}$
Mixture	$T_i = p_i T, \quad E_i = p_i T / D_i, \quad \text{tokens/step from } i = p_i \cdot B$

6 Numerical problems with full worked solutions

Throughout: $N_c = 8.8 \times 10^{13}$, $D_c = 5.4 \times 10^{13}$, $\alpha_N = 0.076$, $\alpha_D = 0.095$, $\alpha_N/\alpha_D = 0.8$. Write $A = (N_c/N)^{0.8}$ (model term) and $B = D_c/D$ (data term), so $L = (A + B)^{0.095}$. The reliable hand method is logarithms: $L = 10^{0.095 \log_{10}(A+B)}$.

Part A — Scaling laws

Problem 1 (the slide example, reverse-engineered). For $N = 10^8$ and $D = 10^7$, verify $A \approx 56\,962$ and $L \approx 4.36$. Then find L at $D = 10^8, 10^9, 10^{10}$ and describe the trend.

Solution

$N_c/N = 8.8 \times 10^5$ and $\log_{10} = 5.9445$, so $A = 10^{0.8 \times 5.9445} = 10^{4.7556} = 56\,962$.

D	$B = D_c/D$	$A + B$	$\log_{10}(A + B)$	$L = 10^{0.095 \log_{10}(A+B)}$
10^7	5.4×10^6	5 456 962	6.7369	4.365
10^8	5.4×10^5	596 962	5.7759	3.538
10^9	54 000	110 962	5.0452	3.015
10^{10}	5 400	62 362	4.7949	2.854
∞	0	56 962	4.7556	2.830

Trend: each $10\times$ in data helps less: -0.83 , then -0.52 , then -0.16 . Once $B \ll A$ (around $D \approx 10^{10}$), the 100M model is capacity-limited and the loss sits near its floor 2.83.

Problem 2. A 1B-parameter model is trained on (a) 10^{10} tokens, (b) 10^{11} tokens. Compute the test losses and the percentage improvement.

Solution

$N_c/N = 8.8 \times 10^4$, $\log_{10} = 4.9445$, so $A = 10^{0.8 \times 4.9445} = 10^{3.9556} = 9028$.

(a) $B = 5400$, $A+B = 14428$, $\log_{10} = 4.1592$, $0.095 \times 4.1592 = 0.39512$, so $L = 10^{0.39512} = \mathbf{2.484}$.

(b) $B = 540$, $A+B = 9568$, $\log_{10} = 3.9808$, $0.095 \times 3.9808 = 0.37818$, so $L = \mathbf{2.389}$.

Improvement: $(2.484 - 2.389)/2.484 = \mathbf{3.8\%}$ for $10\times$ more data. In (a) the data term was already smaller than the model term ($B/A = 0.6$), so the model was mostly capacity-limited.

Problem 3 (limiting floors). Compute the best achievable test loss $L(N, \infty)$ for $N = 10^8$, 10^9 and 1.75×10^{11} (GPT-3 size).

Solution

$L(N, \infty) = (N_c/N)^{0.076}$.

- $N = 10^8$: $\log_{10}(8.8 \times 10^5) = 5.9445$, $\times 0.076 = 0.45178$, so $L = \mathbf{2.830}$.

- $N = 10^9$: $4.9445 \times 0.076 = 0.37578$, so $L = \mathbf{2.376}$.

- $N = 1.75 \times 10^{11}$: $N_c/N = 502.9$, $\log_{10} = 2.7015$, $\times 0.076 = 0.20531$, so $L = \mathbf{1.604}$.

Each $10\times$ in parameters multiplies the floor by $10^{-0.076} = 0.839$ (a 16% reduction).

Problem 4 (data-limited regime). For an effectively infinite model, by what factor does the loss change when the data is (a) doubled, (b) increased $10\times$? (c) How much more data is needed to halve the loss?

Solution

$L(\infty, D) = (D_c/D)^{0.095}$, so $L(kD)/L(D) = k^{-0.095}$.

(a) $2^{-0.095} = e^{-0.095 \ln 2} = e^{-0.06585} = \mathbf{0.936}$ (a 6.4% drop).

(b) $10^{-0.095} = \mathbf{0.804}$ (a 19.6% drop).

(c) $k^{-0.095} = \frac{1}{2} \Rightarrow k = 2^{1/0.095} = 2^{10.53} \approx \mathbf{1475\times}$ more data. Halving the loss by data alone takes about three orders of magnitude, which is why the field went from GB to TB of text.

Problem 5 (inverting the law). A 1B-parameter model must reach test loss (a) 3.0, (b) 2.5, (c) 2.3. How many tokens are needed?

Solution

From $L = (A+B)^{\alpha_D}$: $A+B = L^{1/\alpha_D}$, so $B = L^{1/0.095} - A$ and $D = D_c/B$. Here $A = 9028$ and $1/0.095 = 10.526$.

(a) $3.0^{10.526} = 10^{10.526 \times 0.47712} = 10^{5.0223} = 105276$. $B = 96248$, so $D = 5.4 \times 10^{13}/96248 = \mathbf{5.6 \times 10^8}$ tokens.

(b) $2.5^{10.526} = 10^{10.526 \times 0.39794} = 10^{4.1889} = 15447$. $B = 6419$, so $D = \mathbf{8.4 \times 10^9}$ tokens.

(c) $2.3 < L(10^9, \infty) = 2.376$. **Impossible at any D :** the target is below this model's floor. You need a larger model.

Problem 6 (overfitting penalty and proportional scaling). (a) For $N = 10^9$, $D = 10^{10}$, by what percentage does the loss exceed the infinite-data loss? (b) If a 1B model needs 20B tokens for an acceptable penalty, how many tokens does a 10B model need for the *same* penalty?

Solution

(a) $\frac{L(N, D)}{L(N, \infty)} = \left(1 + \frac{B}{A}\right)^{0.095} = (1 + 5400/9028)^{0.095} = 1.5981^{0.095} = e^{0.095 \times 0.4688} = e^{0.04454} = \mathbf{1.0455}$, i.e. 4.6% above the floor. Check: $2.484/2.376 = 1.045$. ✓

(b) The same penalty needs the same $B/A \propto N^{0.8}/D$, so $D \propto N^{0.8}$ and $D_{10B} = 20B \times 10^{0.8} =$

$20 \times 6.31 = \mathbf{126B}$ tokens.

Problem 7 (GPT-2 vs T5). Use the law to predict the test loss of GPT-2 ($N = 1.5B$, $D = 19B$) and T5 ($N = 11B$, $D = 175B$). Which term dominates in each?

Solution

GPT-2: $N_c/N = 58\,667$, $A = 10^{0.8 \times 4.7684} = 10^{3.8147} = 6\,527$; $B = 5.4 \times 10^{13} / 1.9 \times 10^{10} = 2\,842$; $A + B = 9\,369$, $\log_{10} = 3.9717$, so $L = 10^{0.37731} = \mathbf{2.38}$. The model term dominates ($A/B \approx 2.3$).
T5: $N_c/N = 8\,000$, $A = 10^{0.8 \times 3.9031} = 10^{3.1225} = 1\,326$; $B = 308.6$; $A + B = 1\,634$, $\log_{10} = 3.2133$, so $L = 10^{0.30527} = \mathbf{2.02}$. Still model-dominated ($A/B \approx 4.3$).
 (These are the law's predictions under Kaplan's setup and tokenizer. The real models used different tokenizers and objectives, so do not compare them with published numbers.)

Problem 8 (counting non-embedding parameters). A transformer has $n_{\text{layer}} = 24$ and $d_{\text{model}} = 1024$, with $d_{ff} = 4d_{\text{model}}$. (a) Estimate N . (b) Predict its test loss on 10^{10} tokens.

Solution

(a) Per layer: attention $4d^2$ (W_Q, W_K, W_V, W_O , each $d \times d$) plus MLP $2 \times d \times 4d = 8d^2$, so $12d^2$ in total. $N = 12 \times 24 \times 1024^2 = 12 \times 24 \times 1\,048\,576 = \mathbf{301,989,888} \approx 3.0 \times 10^8$.
 (b) $N_c/N = 2.914 \times 10^5$, $\log_{10} = 5.4645$, $A = 10^{4.3716} = 23\,528$; $B = 5\,400$; $A + B = 28\,928$, $\log_{10} = 4.4613$, so $L = 10^{0.42382} = \mathbf{2.65}$.

Part B — Common Crawl, pipelines and funnels

Problem 9 (CC formats). Using the CC table: (a) Convert the WARC, WAT and WET sizes to TB (decimal). (b) What fraction of WARC size is WET? (c) What is the average compressed size of one WARC file?

Solution

1 TiB = 2^{40} B = 1.0995×10^{12} B.
 (a) WARC: $86.77 \times 1.0995 = \mathbf{95.40}$ TB. WAT: $21.11 \times 1.0995 = \mathbf{23.21}$ TB. WET: $8.69 \times 1.0995 = \mathbf{9.55}$ TB.
 (b) $8.69/86.77 = \mathbf{10.0\%}$. Stripping HTML, headers and scripts leaves about one tenth.
 (c) $86.77 \text{ TiB} / 80\,000 = 86.77 \times 1024 / 80\,000 \text{ GiB} = \mathbf{1.11 \text{ GiB}}$ per file.

Problem 10 (RefinedWeb funnel). Remaining fractions of CC after each stage: $100 \rightarrow 97.76 \rightarrow 96.31 \rightarrow 47.51 \rightarrow 35.97 \rightarrow 30.15 \rightarrow 23.34 \rightarrow 14.50 \rightarrow 11.67$ (%). A snapshot contains 2×10^{12} tokens. (a) How many tokens are in RW-Raw, RW-Filtered and the final RW? (b) Which single step removes the largest *relative* share of its input? (c) Verify the slide's " $\approx 50\%$, $\approx 24\%$, $\approx 12\%$, almost 90% ".

Solution

(a) RW-Raw = $0.4751 \times 2 \times 10^{12} = \mathbf{9.50 \times 10^{11}}$. RW-Filtered = $0.2334 \times 2 \times 10^{12} = \mathbf{4.67 \times 10^{11}}$. RW = $0.1167 \times 2 \times 10^{12} = \mathbf{2.33 \times 10^{11}}$.
 (b) Relative removals: URL 2.24%, extraction 1.48%, **LID 50.67%**, repetition 24.29%, document 16.18%, line 22.59%, fuzzy dedup 37.87%, exact 19.52%. The largest is **language identification** (only English is kept). The largest among cleaning steps proper is **fuzzy dedup** (37.9%).
 (c) In points of CC: LID $96.31 - 47.51 = 48.8 \approx 50$; quality filtering $47.51 - 23.34 = 24.17 \approx 24$; dedup $23.34 - 11.67 = 11.67 \approx 12$; total $100 - 11.67 = 88.33 \approx$ "almost 90% ". ✓

Problem 11 (SETU funnel). Sangraha: $100 \rightarrow 77.95 \rightarrow 56.03 \rightarrow 24.56$ (%). Compute absolute and relative removal at each stage. If 1 TB of prepared documents enters, how much comes out?

Solution

Stage	Remaining	Removed (points)	Removed (relative)
Cleaning & analysis	77.95	22.05	22.05%
Flagging & filtering	56.03	21.92	28.12%
Deduplication	24.56	31.47	56.17%

Output: $0.2456 \times 1 \text{ TB} = \mathbf{245.6 \text{ GB}}$. Dedup removes more than half of what reaches it, the biggest cut, even though the websites were manually verified.

Problem 12 (C4 yield and bytes/token). (a) C3 is 6 TB and C4 is 750 GB. What fraction do the heuristics keep? (b) C4 has $\sim 156\text{B}$ tokens. Bytes per token? (c) The Pile is 825.18 GiB with $\sim 170\text{B}$ tokens. Bytes per token?

Solution

- (a) $0.75/6 = \mathbf{12.5\%}$ kept (87.5% removed).
 (b) $750 \times 10^9 / 156 \times 10^9 = \mathbf{4.81}$ bytes/token.
 (c) $825.18 \times 2^{30} = 8.860 \times 10^{11}$ B, and $8.860 \times 10^{11} / 1.70 \times 10^{11} = \mathbf{5.21}$ bytes/token. This is higher than C4, consistent with the Pile's code, math and non-prose content, which tokenizes less compactly per byte.

Problem 13 (CCNet imbalance). English has $\approx 550\text{B}$ tokens and Hebrew $\approx 1.5\text{B}$ after dedup (Feb 2019). (a) What is the ratio? (b) Tamil has 0.5M Wikipedia pages and CCNet yields 0.9M Tamil documents from that snapshot. Assuming no overlap, how many documents are available in total, and how many snapshots would be needed to reach 10M documents if each contributed 0.9M *new* documents?

Solution

- (a) $550/1.5 \approx \mathbf{367\times}$.
 (b) Total = $0.5 + 0.9 = \mathbf{1.4\text{M}}$ documents. For 10M: $0.5 + 0.9k \geq 10 \Rightarrow k \geq 10.6$, so **11 snapshots**. In reality later snapshots overlap heavily with earlier ones (the same sites are recrawled), so many more are needed. That is why CCNet-style work uses *years* of dumps.

Part C — Deduplication mathematics

Problem 14 (Jaccard and shingle size). $d_1 =$ “the cat sat on the mat”, $d_2 =$ “the cat sat on a mat”. Compute the Jaccard similarity using (a) word 1-grams (sets), (b) word 2-shingles.

Solution

- (a) $S_1 = \{\text{the, cat, sat, on, mat}\}$ (“the” repeats but sets ignore duplicates), $S_2 = \{\text{the, cat, sat, on, a, mat}\}$. $|S_1 \cap S_2| = 5$, $|S_1 \cup S_2| = 6$, so $J = \mathbf{0.833}$.
 (b) $S_1 = \{\text{the cat, cat sat, sat on, on the, the mat}\}$, $S_2 = \{\text{the cat, cat sat, sat on, on a, a mat}\}$. $\cap = \{\text{the cat, cat sat, sat on}\}$ has size 3, $\cup$ has size 7, so $J = 3/7 = \mathbf{0.429}$.
Lesson: longer shingles are *stricter*: one changed word breaks k shingles. The duplicate threshold must be chosen together with k .

Problem 15 (the Aditya-L1 pair from the slide). Lower-case, drop punctuation, and compute J for word 3-shingles.

Solution

d_1 (24 words): “the orbit of aditya-l1 spacecraft is a periodic halo orbit which is located roughly 1.5 million km from earth on the continuously moving sun”.

d_2 (25 words): “... periodic halo orbit the orbit is located roughly ...”.

$|S_1| = 24 - 2 = 22$ and $|S_2| = 25 - 2 = 23$ shingles.

Only in d_1 : (halo orbit which), (orbit which is), (which is located), i.e. 3 shingles.

Only in d_2 : (halo orbit the), (orbit the orbit), (the orbit is), (orbit is located), i.e. 4 shingles.

$|S_1 \cap S_2| = 22 - 3 = 19$ and $|S_1 \cup S_2| = 19 + 3 + 4 = 26$, so $J = 19/26 = \mathbf{0.731}$. (With word 1-grams: $20/21 = 0.952$.) With a typical threshold of 0.7–0.8 this pair is borderline under 3-shingles, and clearly a duplicate under 1-grams.

Problem 16 (MinHash by hand). Universe $\{0, \dots, 5\}$, $A = \{0, 2, 3\}$, $B = \{0, 1, 3, 4\}$. Five random permutations π_j are given as lists $[\pi(0), \dots, \pi(5)]$: $\pi_1 = [0, 2, 3, 5, 4, 1]$, $\pi_2 = [1, 3, 2, 0, 4, 5]$, $\pi_3 = [4, 5, 0, 1, 2, 3]$, $\pi_4 = [2, 0, 1, 3, 5, 4]$, $\pi_5 = [4, 0, 2, 3, 5, 1]$. Compute the MinHash signatures, $\hat{J}$, and compare with the true J .

Solution

True: $A \cap B = \{0, 3\}$ and $A \cup B = \{0, 1, 2, 3, 4\}$, so $J = 2/5 = 0.4$.

j	π_j on $A = \{0, 2, 3\}$	min	π_j on $B = \{0, 1, 3, 4\}$	min	match?
1	0,3,5	0	0,2,5,4	0	✓
2	1,2,0	0	1,3,0,4	0	✓
3	4,0,1	0	4,5,1,2	1	×
4	2,1,3	1	2,0,3,5	0	×
5	4,2,3	2	4,0,3,5	0	×

Signatures: $\mathbf{m}(A) = (0, 0, 0, 1, 2)$ and $\mathbf{m}(B) = (0, 0, 1, 0, 0)$. They agree in 2 of 5 positions, so $\hat{J} = \mathbf{0.4} = J$. In rows 1–2 the minimising element (0, then 3) lies in $A \cap B$. In rows 3–5 it lies in $A \setminus B$ (element 2) or $B \setminus A$ (element 1), exactly as the proof predicts. Standard error with $k = 5$: $\sqrt{0.4 \times 0.6/5} = 0.22$, so this exact hit was partly luck.

Problem 17 (how many hash functions?). How many MinHash functions k are needed so that the standard error of $\hat{J}$ is at most 0.02 in the worst case?

Solution

$\text{SE} = \sqrt{J(1-J)/k}$, maximised at $J = 0.5$: $\sqrt{0.25/k} \leq 0.02 \Rightarrow k \geq 0.25/0.0004 = \mathbf{625}$.

Problem 18 (LSH banding). A MinHash signature of length $k = 100$ is split into $b = 20$ bands of $r = 5$ rows. (a) Find the probability that pairs with $s = 0.3, 0.5, 0.8$ become candidates. (b) Find the threshold. (c) Propose (b, r) with $br = 100$ whose threshold is near 0.8 and evaluate it at $s = 0.5, 0.8, 0.9$.

Solution

$$P = 1 - (1 - s^r)^b.$$

(a) $s = 0.3$: $0.3^5 = 0.00243$, $(0.99757)^{20} = 0.9525$, so $P = \mathbf{0.047}$.

$s = 0.5$: $0.5^5 = 0.03125$, $(0.96875)^{20} = 0.530$, so $P = \mathbf{0.470}$.

$s = 0.8$: $0.8^5 = 0.32768$, $(0.67232)^{20} = 3.6 \times 10^{-4}$, so $P = \mathbf{0.9996}$.

(b) $s^* \approx (1/20)^{1/5} = e^{-\ln 20/5} = e^{-0.599} = \mathbf{0.55}$.

(c) $b = 10, r = 10$: $s^* = (0.1)^{0.1} = \mathbf{0.79}$.

$s = 0.5$: $1 - (1 - 0.000977)^{10} = \mathbf{0.0097}$.

$s = 0.8$: $0.8^{10} = 0.1074$, $1 - (0.8926)^{10} = 1 - 0.321 = \mathbf{0.679}$.

$s = 0.9$: $0.9^{10} = 0.3487$, $1 - (0.6513)^{10} = \mathbf{0.986}$.

Increasing r sharpens rejection of dissimilar pairs but misses some true duplicates near the threshold.

Problem 19 (hash collisions in exact dedup). Exact line dedup over $n = 10^9$ distinct lines. Expected number of false collisions with (a) 32-bit, (b) 64-bit hashes?

Solution

$\mathbb{E} \approx n^2/2^{b+1}$.

(a) $10^{18}/2^{33} = 10^{18}/8.59 \times 10^9 = \mathbf{1.16 \times 10^8}$ collisions. Unacceptable: over 100M lines would be wrongly deleted.

(b) $10^{18}/2^{65} = 10^{18}/3.69 \times 10^{19} = \mathbf{0.027}$. Negligible.

Problem 20 (suffix array). Build the suffix array and LCP array for $S = \text{abcab\$}$ and find the longest repeated substring.

Solution

Suffixes: 0 $\text{abcab\$}$, 1 $\text{bcab\$}$, 2 $\text{cab\$}$, 3 $\text{ab\$}$, 4 $\text{b\$}$, 5 $\text{\$}$.

rank	SA	suffix	LCP with previous
0	5	$\text{\$}$	—
1	3	$\text{ab\$}$	0
2	0	$\text{abcab\$}$	2 (ab)
3	4	$\text{b\$}$	0
4	1	$\text{bcab\$}$	1 (b)
5	2	$\text{cab\$}$	0

$SA = [5, 3, 0, 4, 1, 2]$. The maximum LCP is 2, so the longest repeated substring is **ab** (positions 0 and 3). In dataset dedup the same scan flags every repeated span above the length threshold (e.g. 50 tokens).

Problem 21 (memorisation vs. duplication). A sequence appearing 10 times is generated $\sim 1000\times$ more often than one appearing once. Assuming $g(k) \propto k^\gamma$: (a) find γ ; (b) predict the ratio for 5 copies vs. 1; (c) if dedup reduces a sequence from 40 copies to 2, by what factor does its generation rate fall?

Solution

(a) $10^\gamma = 1000 \Rightarrow \gamma = \mathbf{3}$. (b) $5^3 = \mathbf{125\times}$. (c) $(40/2)^3 = 20^3 = \mathbf{8000\times}$ lower. (This power law is an illustrative fit to one data point; the real relation from Kandpal et al. is empirical.)

Problem 22 (repeated-data experiment). A model is trained on $T = 100\text{B}$ tokens in which 0.1% of the underlying data is repeated 100 times and the rest is seen once, with 90% of training tokens unique. How many distinct tokens are in the repeated subset, and what fraction of the unique data is it?

Solution

Repeated tokens make up 10% of T , i.e. 10B token-occurrences $= 100 \times r$, so $r = \mathbf{100\text{M}}$ distinct tokens. Unique data $= 90\text{B} + 0.1\text{B} = 90.1\text{B}$, so $r/90.1\text{B} = \mathbf{0.11\%}$, consistent with “0.1% of the data”. A tiny subset (100M tokens) consumes 10% of the compute and, per Hernandez et al.,

can degrade an 800M model to 400M-level performance.

Part D — Data mixing

Problem 23 (training budget and per-step mix). $15\text{M steps} \times 2^{16}$ tokens per step with the slide's mix (C4 40%, OpenWebText 15%, Books 10%, RealNews 10%, Codes 10%, Wikipedia 10%, Stories 5%). (a) Find T . (b) Find the expected tokens per step from each source. (c) If Wikipedia has 24B tokens, how many epochs of Wikipedia?

Solution

- (a) $T = 15 \times 10^6 \times 65\,536 = \mathbf{9.83 \times 10^{11}} \approx 1\text{T}$.
 (b) Per step: C4 $0.40 \times 65\,536 = \mathbf{26\,214}$; OpenWebText $\mathbf{9\,830}$; Books, RealNews, Codes, Wikipedia $\mathbf{6\,554}$ each; Stories $\mathbf{3\,277}$. Sum = $65\,536$ ✓
 (c) $E = 0.10 \times 9.83 \times 10^{11} / 2.4 \times 10^{10} = \mathbf{4.1}$ epochs. That is on the high side given Module 1's overfitting warning.

Problem 24 (GPT-3 epochs). Using GPT-3's table (300B training tokens), compute epochs per source.

Solution

$E_i = p_i \times 300 / D_i$ (in billions):

Source	D_i (B)	p_i	E_i
Common Crawl	410	0.60	$180/410 = \mathbf{0.44}$
WebText2	19	0.22	$66/19 = \mathbf{3.47}$
Books1	12	0.08	$24/12 = \mathbf{2.00}$
Books2	55	0.08	$24/55 = \mathbf{0.44}$
Wikipedia	3	0.03	$9/3 = \mathbf{3.00}$

Less than half of filtered CC is ever seen, while WebText2 and Wikipedia are repeated about 3–3.5 times. That is deliberate up-weighting of higher-quality sources.

Problem 25 (backing out source sizes from LLaMA). LLaMA trained on $\sim 1.4\text{T}$ tokens. Using sampling proportions and epochs, estimate the unique token count of Wikipedia (4.5%, 2.45 epochs), Books (4.5%, 2.23), GitHub (4.5%, 0.64) and CommonCrawl (67%, 1.10).

Solution

$D_i = p_i T / E_i$: Wikipedia $0.045 \times 1400 / 2.45 = \mathbf{25.7\text{B}}$; Books $63 / 2.23 = \mathbf{28.3\text{B}}$; GitHub $63 / 0.64 = \mathbf{98.4\text{B}}$; CommonCrawl $938 / 1.10 = \mathbf{853\text{B}}$. Same share (4.5%) but very different epochs: small sources get repeated, while GitHub is under-sampled (less than one pass).

Problem 26 (composite-dataset arithmetic). (a) Sum RedPajama-V1's components. (b) What fraction did SlimPajama's dedup remove? (c) What fraction of DOLMA is web pages (Common Crawl + C4)?

Solution

- (a) $878 + 175 + 59 + 26 + 28 + 24 + 20 = \mathbf{1210\text{B}} \approx 1.2\text{T}$. (b) $1 - 670/1210 = \mathbf{44.6\%}$ removed.
 (c) $(2281 + 198) / 3059.3 = \mathbf{81.0\%}$.

Problem 27 (designing a mixture). You have 1T tokens of budget and sources: Web 2T tokens, Code 200B, Books 30B, Wikipedia 20B. You want at most 3 epochs on any source and want Books and Wikipedia at 3 epochs. Find $p_{\text{Books}}, p_{\text{Wiki}}$, then split the rest between Web and Code so that Code gets 1 epoch. Check all epoch limits.

Solution

Books: $p \cdot 1000 = 3 \times 30 = 90 \Rightarrow p_{\text{Books}} = 0.09$. Wiki: $3 \times 20 = 60 \Rightarrow p_{\text{Wiki}} = 0.06$. Code at 1 epoch: $200 \Rightarrow p_{\text{Code}} = 0.20$. Web: $1 - 0.09 - 0.06 - 0.20 = \mathbf{0.65}$, i.e. 650B tokens, which is $650/2000 = 0.325$ epochs. Final: Web 65% (0.33 ep), Code 20% (1 ep), Books 9% (3 ep), Wiki 6% (3 ep). All $\leq 3 \checkmark$. This mirrors the LLaMA/GPT-3 pattern.

7 Subjective questions with model answers

Q1. Explain the two pieces of evidence the lecture gives for scaling the dataset size.

Answer. (1) *T5's repetition study*: with training steps fixed, shrinking the corpus forces more epochs. Performance degrades and training loss falls suspiciously low, so the model overfits and memorises repeated data. Fresh data is better than repeated data. (2) *Kaplan scaling law*: $L(N, D) = [(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$. Test loss falls monotonically with D , and the data term is a separate bottleneck that no amount of parameters removes ($L(\infty, D) = (D_c/D)^{\alpha_D}$). Because bigger models are more data-hungry ($D \propto N^{0.8}$ for a constant overfitting penalty), scaling parameters from 1.5B to 175B+ requires scaling data proportionally.

Q2. Why does the scaling law use *non-embedding* parameters, and why the exponent α_N/α_D inside the bracket?

Answer. Embedding matrices ($|V| \times d$) can be a large share of the parameters of small models but add little per-token computation. Including them blurs the power-law trends, and excluding them gives clean, architecture-independent fits. The exponent α_N/α_D is chosen so that, after the outer power α_D , the $D \rightarrow \infty$ limit is exactly the separate model-size law $(N_c/N)^{\alpha_N}$, and the $N \rightarrow \infty$ limit is exactly $(D_c/D)^{\alpha_D}$. The combined formula then interpolates between two independently validated power laws.

Q3. The slide says a 100M model on 10M tokens has training loss $\rightarrow 0$ but test loss 4.36. Explain this gap and what it tells a practitioner.

Answer. With 10 parameters per training token, the model can memorise the training set, so training loss goes to about 0. The scaling law predicts *generalisation*: the data term (5.4×10^6) is $\sim 95\times$ the model term (5.7×10^4), so the model is massively data-starved. The gap is pure overfitting. The practitioner should add data, not parameters. Adding parameters would barely move the test loss because the bottleneck is D .

Q4. Describe the three axes of data and give an example of a dataset or decision that trades one axis against another.

Answer. **Scale** (tokens), **quality** (clean, informative, non-toxic, deduplicated), **diversity** (domains, languages, registers). Examples: RefinedWeb trades scale for quality, discarding $\sim 88\%$ of a CC snapshot. Gopher refused to use a Wikipedia/Reddit-trained quality classifier because optimising “quality” that way would cut diversity (dialects, sociolects). “Textbooks Are All You Need” pushes quality at small scale. The Pile/DOLMA explicitly buy diversity by combining 7–22 sources.

Q5. Compare WARC, WAT and WET. Why might a pipeline deliberately start from the much larger WARC files?

Answer. WARC holds full HTTP responses including raw HTML (86.77 TiB). WAT holds computed metadata such as links, headers and titles (21.11 TiB). WET holds plain text extracted by Common Crawl (8.69 TiB, $\approx 10\%$ of WARC). WET is cheap but its extraction is crude: navigation, menus and footers stay in. Starting from WARC lets the pipeline use a better extractor (jusText in Pile-CC, Trafilatura in RefinedWeb/Sangraha) that uses HTML structure to separate main content from boilerplate. You pay for cleaner text with $\sim 10\times$ more I/O.

Q6. List the five mechanisms a pre-processing pipeline needs and a typical tool for each. Which ones do the fastText basic pipeline and CCNet satisfy?

Answer. (1) Language detection: fastText, langdetect, pylid2, IndicLID. (2) Deduplication: hashing, suffix arrays, MinHash/SimHash/LSH. (3) Toxicity: bad-word lists, LM classifiers, Perspective API. (4) Quality: reference-LM perplexity/classifiers (Wikipedia, books), heuristics. (5) PII: regex. The fastText basic pipeline does (1) and (2) with line-level dedup. CCNet does (1), (2) (paragraph-level) and (4). Neither handles toxicity or PII.

Q7. Explain the difference between exact and fuzzy deduplication using the Aditya-L1 example, and name the algorithms for each.

Answer. Exact duplicates are byte-identical: webpage-1 and webpage-2 both contain “The orbit of Aditya-L1 spacecraft is a periodic Halo orbit”. Hashing or a suffix array finds them. Fuzzy (near) duplicates differ trivially: “...Halo orbit *which* is located ...” vs. “...Halo orbit. *The orbit* is located ...”. Their hashes differ completely, but their shingle sets overlap heavily ($J \approx 0.73$ on word 3-shingles). Fuzzy methods (MinHash with LSH, SimHash) estimate set or cosine similarity cheaply and flag pairs above a threshold.

Q8. Prove that MinHash collision probability equals Jaccard similarity, and explain how LSH banding avoids $O(n^2)$ comparisons.

Answer. Under a random permutation, the minimum over $A \cup B$ is equally likely to be any of its $|A \cup B|$ elements. $\min(A) = \min(B)$ iff that element lies in $A \cap B$, so the probability is $|A \cap B|/|A \cup B| = J$. With $k = br$ hashes split into b bands of r rows, each band is hashed to a bucket, and only documents sharing a bucket in some band are compared. A pair with similarity s becomes a candidate with probability $1 - (1 - s^r)^b$, an S-curve with threshold $\approx (1/b)^{1/r}$. Dissimilar pairs almost never share buckets, so work is roughly linear in n plus the (small) number of candidate pairs.

Q9. Walk through the CCNet pipeline and explain why it matters for low-resource languages such as Tamil.

Answer. WET $\rightarrow$ paragraph-level hash dedup (removes boilerplate repeated across pages) $\rightarrow$ fastText LID $\rightarrow$ quality filtering against a Wikipedia-trained LM (head/middle/tail by perplexity; low-quality removed by threshold) $\rightarrow$ JSON. It produced data for 130 languages. For Tamil, Wikipedia has only 0.5M pages (vs. 59M for English), but CCNet on a single Feb 2019 snapshot yields 0.9M Tamil documents, and years of snapshots multiply this. Without CC-based pipelines, low-resource languages lack enough monolingual pre-training text.

Q10. Describe C4’s filtering heuristics and critique them.

Answer. Keep lines ending in terminal punctuation; drop pages with fewer than 5 sentences and lines with fewer than 3 words; remove pages with bad words, “lorem ipsum” or curly brackets; drop lines mentioning JavaScript or cookie/policy boilerplate; remove citation markers; dedup 3-sentence spans; keep only pages with $P(\text{en}) \geq 0.99$. *Strengths:* cheap, transparent, and effective against menus, placeholders and code. *Weaknesses:* blunt. The curly-bracket rule deletes all code-containing pages. The bad-word list removes non-toxic medical text and text by or about marginalised groups (a diversity and fairness cost). The terminal-punctuation rule drops valid list-style content. And exact 3-sentence dedup misses near-duplicates.

Q11. Using RefinedWeb’s numbers, explain how “almost 90%” of a CC snapshot gets removed and argue whether this is wasteful.

Answer. URL filtering and extraction remove ~ 3.7 points, English-only LID removes 48.8, quality filtering (repetition, document, line) removes 24.2, and dedup (MinHash then exact substring) removes 11.7, leaving 11.67%. It is not wasteful: the removed content is non-English (useful elsewhere), boilerplate, spam, or duplicates, which Module 2 shows are *harmful* (memorisation, contamination). Because CC has many snapshots, discarding 88% of each still yields ~ 5 T tokens. The Falcon results show the filtered web alone can match curated corpora. The scaling law rewards unique, clean tokens.

Q12. Why is deduplication important? Include the quantitative evidence from the lecture.

Answer. (i) Memorisation: a sequence present 10 times is generated $\sim 1000\times$ more often than one present once. (ii) Misleading evaluation: C4 has 1.38% and RealNews 3.37% of validation examples with duplicates in train, so test scores overstate generalisation. (iii) Privacy: memorised PII can be extracted. (iv) Wasted compute and degraded quality: repeating 0.1% of data 100 times degraded an 800M model to 400M-level. Larger models are more sensitive: harmful at a hundred duplicates for 1B parameters, at a few for 175B. Counterpoint: Pythia found deduplicating the Pile had limited zero-shot impact, so the size of the effect depends on setting and metric.

Q13. “All pipelines are leaky.” Discuss with reference to OPT, Gopher and the Pretrainer’s Guide.

Answer. OPT found the Pile “particularly full of duplicate documents” despite its curation. Pipelines miss duplicates. Gopher avoided classifier-based quality filtering because a “gold” reference (Wikipedia, Reddit-linked pages) biases towards certain demographics and erases dialects. Filters can remove good data. The Pretrainer’s Guide notes that quality is context-dependent: a high-quality Reddit post doesn’t look like a high-quality paper, and peer review itself is noisy. Every filter encodes assumptions. The result is under-filtering (duplicates, toxicity leak through) and over-filtering (valid but unusual text removed) at the same time.

Q14. How does the SETU pipeline differ from CC-based English pipelines, and why?

Answer. SETU (for Sangraha) draws on three sources: manually verified web domains (scraped with Trafilatura), *videos transcribed by ASR*, and *digitised PDFs via OCR*, including synthetic data. It uses Indic-specialised LID (IndicLID, CLD3, NLLB), MinHash fuzzy dedup, and runs on Spark. Indian languages have little clean web text but a great deal of speech and print, so extra modalities are needed to reach scale. Verification counters the high share of low-quality or machine-translated Indic web pages. Even so, only $\approx 24.6\%$ survives, and dedup is the largest cut.

Q15. Explain how data mixing works during pre-training and why sampling proportions differ from dataset sizes.

Answer. Each training step’s batch (e.g. 2^{16} tokens) is assembled by sampling sources with fixed probabilities p_i . Over T tokens, source i contributes $p_i T$ tokens, i.e. $E_i = p_i T / D_i$ epochs. If $p_i \propto D_i$, huge noisy sources (CC) would dominate and small high-quality ones (Wikipedia, books) would be negligible. So designers up-weight high-quality sources (LLaMA: Wikipedia 2.45 epochs, Books 2.23) and down-weight or partially use others (GPT-3 CC 0.44 epochs, LLaMA GitHub 0.64), while keeping repetition modest to avoid Module 1’s overfitting. The mixture also shapes capabilities: dialog-heavy for LaMDA/PaLM, academic for Minerva/Galactica, code for AlphaCode.

Q16. Compare The Pile, RedPajama-V1 and DOLMA. What trend do they show?

Answer. The Pile (2020): 22 curated sources, 825 GiB (~ 170 B tokens). Pile-CC is the largest part (227 GiB), followed by books, PubMed, GitHub, arXiv, law and more. It is built for diversity. RedPajama-

V1 (2023): an open replication of LLaMA’s recipe, 1.2T tokens (CC 878B, C4 175B, GitHub, Books, ArXiv, Wikipedia, StackExchange); dedup to SlimPajama leaves 670B. DOLMA (2024): 3.06T Llama tokens (CC 2281B, The Stack 411B, C4, Reddit, PeS2o, Gutenberg, Wikipedia), fully open, used for OLMo. Trend: token counts grow $\sim 18\times$ in four years; web remains the majority ($\sim 81\%$ of DOLMA); openness and documentation improve; code becomes a standard ingredient.

Q17. Discuss the “questions to ponder”: data limits, synthetic data, small high-quality datasets, and mixing code with text.

Answer. *Limits:* high-quality unique human text is finite. Frontier runs consume a large fraction of filtered web tokens, so the data term of the scaling law becomes binding. *Synthetic data:* LLM-generated data (phi-1’s textbooks, translated Indic data) can add targeted, high-quality tokens, but risks inherited errors, low diversity and model collapse under recursive training. It works best when filtered and mixed with real data. *Small high-quality data:* 30–100B curated tokens can give strong small or specialised models (sample efficiency), but cannot remove the data bottleneck for very large models. *Code + text:* code is precise and structured. Moderate amounts tend to improve reasoning and structured generation, while excessive amounts dilute natural-language skill. It comes down to choosing p_{code} .

8 Multiple-choice questions

Try each question before reading the answer key in Section 8.1.

1. In the scaling law $L(N, D) = [(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$, what does N count?
 - (a) Number of training steps
 - (b) Non-embedding parameters
 - (c) All model parameters
 - (d) Embedding parameters only
2. The value of α_N/α_D used on the slide is
 - (a) 0.095
 - (b) 1.25
 - (c) 0.076
 - (d) 0.8
3. For $N = 10^8$ and $D = 10^7$, the predicted test loss is approximately
 - (a) 4.36
 - (b) 2.83
 - (c) 3.54
 - (d) 0.0
4. As $D \rightarrow \infty$ with N fixed, $L(N, D)$ tends to
 - (a) 0
 - (b) $(D_c/D)^{\alpha_D}$
 - (c) $(N_c/N)^{\alpha_N/\alpha_D}$
 - (d) $(N_c/N)^{\alpha_N}$
5. In the data-limited regime, doubling the dataset multiplies the loss by about
 - (a) 0.50
 - (b) 0.990
 - (c) 0.80
 - (d) 0.936
6. To keep the relative overfitting penalty constant, D should scale with N as

- (a) $D \propto N^{1.25}$
 - (b) $D \propto N^{0.095}$
 - (c) $D \propto N^{0.8}$
 - (d) $D \propto N$
7. The T5 study on repetition showed that
- (a) only code data can be repeated safely
 - (b) repeating data many times improves downstream accuracy
 - (c) repeating data for many epochs leads to overfitting
 - (d) repetition has no effect for large models
8. Which is *not* one of the three axes of data in the lecture?
- (a) Quality
 - (b) Latency
 - (c) Diversity
 - (d) Scale
9. Which Common Crawl format contains plain extracted text?
- (a) WET
 - (b) WARC
 - (c) WAT
 - (d) CDX
10. The approximate compressed size of the WARC files of one CC snapshot is
- (a) 21.11 TiB
 - (b) 8.69 TiB
 - (c) 2.34 TiB
 - (d) 86.77 TiB
11. Which tool is specifically designed for Indian-language identification?
- (a) Perspective API
 - (b) langdetect
 - (c) pylid2
 - (d) IndicLID
12. PII removal in the pipelines discussed is typically done with
- (a) Regex patterns
 - (b) MinHash
 - (c) jusText
 - (d) KenLM perplexity
13. The fastText basic pipeline uses deduplication at the
- (a) line level
 - (b) document level
 - (c) paragraph level
 - (d) 3-sentence-span level
14. CCNet performs deduplication at the
- (a) URL level
 - (b) character level
 - (c) paragraph level
 - (d) line level

- 15.** Which requirements does CCNet satisfy?
- (a) LID and dedup only
 - (b) All five
 - (c) LID, dedup and quality filtering
 - (d) LID, dedup, toxicity and PII
- 16.** In CCNet, the quality score comes from
- (a) a bad-word list
 - (b) human annotators
 - (c) a language model pre-trained on a targeted domain such as Wikipedia
 - (d) URL blocklists
- 17.** C4 removes any page that contains
- (a) more than 5 sentences
 - (b) a Wikipedia link
 - (c) “lorem ipsum” or a curly bracket
 - (d) a full stop
- 18.** In the C4 pipeline figure, the dataset produced by LID + deduplication only (no heuristics) is
- (a) C4 (750 GB)
 - (b) WebText-like (17 GB)
 - (c) RealNews-like (35 GB)
 - (d) C3 (6 TB)
- 19.** Pile-CC differs from C4 in that it
- (a) uses no language detection
 - (b) uses WET files and exact dedup
 - (c) is multilingual
 - (d) uses WARC files, jusText extraction and fuzzy dedup
- 20.** RefinedWeb extracts text with
- (a) Trafilatura
 - (b) BeautifulSoup
 - (c) jusText
 - (d) CC’s WET extraction
- 21.** Roughly what fraction of a single CC snapshot survives the RefinedWeb pipeline?
- (a) ~50%
 - (b) ~12%
 - (c) ~90%
 - (d) ~24%
- 22.** In RefinedWeb, which step removes the most data (in points of CC)?
- (a) Language identification
 - (b) Exact deduplication
 - (c) URL filtering
 - (d) Fuzzy deduplication
- 23.** The Sangraha/SETU pipeline uses which dedup method?
- (a) None
 - (b) SimHash
 - (c) Exact suffix-array

(d) Fuzzy MinHash

24. Which pair of extra modalities does SETU use?

- (a) Images and captions
- (b) Tables and HTML
- (c) ASR on videos and OCR on PDFs
- (d) Code and math

25. For sets A, B and a random hash, $P[\min h(A) = \min h(B)]$ equals

- (a) $|A \cap B|/|A|$
- (b) $|A \cap B|/|A \cup B|$
- (c) cosine similarity
- (d) $1 - \theta/\pi$

26. With LSH banding (b bands, r rows), the candidate probability for similarity s is

- (a) $(1 - s^b)^r$
- (b) $1 - (1 - s^r)^b$
- (c) $1 - (1 - s^b)^r$
- (d) s^{br}

27. A single 61-word sentence was found repeated about how many times in C4?

- (a) 600
- (b) 6 000
- (c) 60 000
- (d) 600 000

28. A sequence present 10 times in training is generated on average how much more often than one present once?

- (a) $10\times$
- (b) $1000\times$
- (c) No difference
- (d) $100\times$

29. Repeating 0.1% of the data 100 times degraded an 800M model to the performance of a

- (a) 100M model
- (b) 400M model
- (c) 200M model
- (d) 1.6B model

30. Gopher avoided a Wikipedia/Reddit-based quality classifier because

- (a) it could bias towards certain demographics and erase dialects
- (b) Wikipedia is too small
- (c) it was too slow
- (d) it removed too little data

31. Which dataset is described as having 22 sources?

- (a) mC4
- (b) The Pile
- (c) C4
- (d) RefinedWeb

32. RedPajama-V1 after extensive dedup is called

- (a) SlimPajama
- (b) RedPajama-v2
- (c) RefinedPajama
- (d) TinyPajama

33. In LLaMA’s mixture, which source was seen for *less* than one epoch?

- (a) Wikipedia
- (b) Books
- (c) ArXiv
- (d) GitHub

34. With 15M steps of 2^{16} tokens each, the total training tokens are about

- (a) 10^{15}
- (b) 10^{10}
- (c) 10^{12}
- (d) 10^9

35. Which models gave about 50% weight to dialog/conversation data?

- (a) BERT and GPT-2
- (b) Minerva and Galactica
- (c) LaMDA and PaLM
- (d) AlphaCode and CodeGen

36. Which dataset is *multilingual*?

- (a) ROOTS
- (b) C4
- (c) RefinedWeb
- (d) The Pile

37. Why is comparing datasets by tokens often preferred over storage size?

- (a) Tokens are always fewer
- (b) Tokenizers are identical across models
- (c) Storage cannot be measured
- (d) Tokens relate directly to the scaling law’s D

38. In GPT-3’s mixture, Common Crawl had weight 60% of 300B tokens and 410B tokens available. Epochs over CC $\approx$

- (a) 2.0
- (b) 0.44
- (c) 1.0
- (d) 3.47

39. Which statement about duplicates in The Pile is supported by the slide footnote?

- (a) The Pile was deduplicated with suffix arrays only
- (b) Pythia found deduplicating The Pile had limited impact on zero-shot performance
- (c) The Pile contains no duplicates
- (d) Deduplicating it always hugely improves zero-shot scores

40. “Is mixing code (non-ambiguous) and text (ambiguous) helpful or harmful?” is posed as

- (a) an open question to ponder
- (b) a C4 heuristic
- (c) a CCNet design choice

- (d) a solved result: always harmful

8.1 Answer key with explanations

1. (b) Kaplan et al. exclude embedding parameters for cleaner power laws.
2. (d) $0.076/0.095 = 0.8$.
3. (a) $[56\,962 + 5.4 \times 10^6]^{0.095} \approx 4.36$. (d) is the *training* loss; (b) is the infinite-data floor.
4. (d) The data term vanishes, and $[(N_c/N)^{0.8}]^{0.095} = (N_c/N)^{0.076}$.
5. (d) $2^{-0.095} \approx 0.936$.
6. (c) Keep $B/A = (D_c/D)(N/N_c)^{0.8}$ constant.
7. (c) Many repetitions lead to overfitting and memorisation, hence the need to scale data.
8. (b) The axes are scale, quality and diversity.
9. (a) WET = WARC Encapsulated Text (“Web Extracted Text”).
10. (d) WARC 86.77, WAT 21.11, WET 8.69 TiB.
11. (d) IndicLID (AI4Bharat) covers Indic scripts and romanised text.
12. (a) Emails, phone numbers and IPs are caught by regex.
13. (a) “Uses line level deduplication”.
14. (c) Paragraph-level hash dedup.
15. (c) Dedup, LID and targeted quality filtering: three requirements.
16. (c) An LM trained on the target domain (Wikipedia), i.e. perplexity-based.
17. (c) Placeholder text and code indicators.
18. (d) The C3 branch is dedup only, 6 TB.
19. (d) WARC + jusText + pyclld2 + fuzzy dedup.
20. (a) Trafilatura (plus LID).
21. (b) 11.67% remains, i.e. almost 90% removed.
22. (a) LID removes 48.8 points (non-English).
23. (d) Fuzzy (MinHash).
24. (c) Videos via ASR, PDFs via OCR.
25. (b) The MinHash property equals Jaccard. (d) is SimHash’s bit-agreement.
26. (b) A band matches w.p. s^r ; at least one of b bands matches.
27. (c) 60 000 repetitions (Lee et al.).
28. (b) $\sim 1000\times$, super-linear.
29. (b) 400M: a $2\times$ smaller model.
30. (a) Bias and erasure of dialects/sociolects.
31. (b) The Pile: 22 diverse sources.
32. (a) SlimPajama (670B tokens).
33. (d) GitHub at 0.64 epochs.
34. (c) $15 \times 10^6 \times 65\,536 \approx 9.8 \times 10^{11} \approx 1\text{T}$.
35. (c) 50% dialog each.
36. (a) ROOTS (46 natural + 13 programming languages).
37. (d) The scaling law is stated in tokens; bytes depend on script and encoding.
38. (b) $0.6 \times 300/410 = 0.44$.
39. (b) Pythia’s finding is the footnote.
40. (a) It is one of the “questions to ponder”.